Interpolable Nyquist criterion: fast stability charts and rightmost-root estimation for linear time-delay systems via adaptive phase integration

Daniel Bachrathy[a]

[a]*Department of Applied Mechanics, Faculty of Mechanical Engineering, Budapest University of Technology and Economics, Műegyetem rkp. 5., H-1111 Budapest, Hungary*

Abstract

A fast method is presented for computing stability charts of linear autonomous time-delay systems. The number of unstable characteristic roots is obtained from the argument principle by tracking the phase of the characteristic function along the imaginary axis, with the phase slope evaluated exactly by forward-mode automatic differentiation. The winding number is either integrated as an ordinary differential equation by an adaptive high-order integrator or, without integration, obtained by discrete phase unwrapping: the exact phase increments between adaptively placed frequency samples are summed under an ODE-style step-size control driven by the exact slope. The integration-free march is 38 to 298 times faster on three benchmarks and more robust near the stability boundary. Since the parameter points are independent, its cost of 4 to 20 microseconds per point on one CPU thread divides almost exactly among CPU threads and, on graphics processors, in proportion to the number of threads up to the saturation of the device: on the fastest of four GPUs measured, a full-HD chart is computed in under a millisecond and an 8K chart in about ten milliseconds, fast enough to recompute it while a parameter slider is being moved; the same kernel runs in any web browser (WebGPU) and on rented cloud GPUs, where brute-force scans of four- to six-dimensional parameter spaces take minutes to hours. During the same sweep, the characteristic root closest to the integration line is estimated as a by-product; it makes the chart interpolable, showing both the number of unstable roots and the stability margin, and drives the multi-dimensional bisection method to trace the true stability boundary. The characteristic function can be extracted automatically from a simulation-ready right-hand side, including singular mass matrices. Retarded, neutral, distributed-delay, transcendental, high-dimensional and fractional-order case studies use one workflow, available in the open-source Julia package `InterpolatedNyquist.jl`.

Keywords: time-delay systems, stability chart, Nyquist criterion, argument principle, neutral delay differential equation, differential–algebraic equation, automatic differentiation, adaptive integration, phase unwrapping, GPU

1. Introduction

Time delays are inherent in a wide range of engineering systems: regenerative machine-tool vibrations [1, 2, 3, 4], digitally controlled and teleoperated machines, active vibration suppression, wheel shimmy, and traffic or population

Email address: bachrathy@mm.bme.hu (Daniel Bachrathy)

dynamics all lead to delay differential equations (DDEs) [5, 6, 7, 8]. Since the characteristic function $\mathcal{D}(\lambda)$ of even the simplest linear autonomous DDE is transcendental, it possesses infinitely many characteristic roots, and the stability of an equilibrium can rarely be decided in closed form. Engineering practice therefore relies on *stability charts*: maps over a plane of design parameters that show where the equilibrium is asymptotically stable [9, 10]. The practical value of such a chart depends on two properties of the underlying numerical method: how fast a single parameter point can be classified, and how much extra information (dominant root location, stability margin, robustness) the classification yields.

A large family of methods operates in the time domain by discretizing either the solution operator or the infinitesimal generator of the delay system. The semi-discretization method of Insperger and Stépán [11, 12, 13, 9], the full-discretization method [14], Chebyshev collocation [15], pseudospectral differencing [16, 17], and the spectral element method [18] all construct a finite transition matrix whose eigenvalues approximate the characteristic multipliers or exponents; spectral methods to certify all roots in a prescribed right half-plane are also available [19]. Mature software packages implement these ideas: DDE-BIFTOOL [20, 21], TRACE-DDE [22], Knut [23, 24], and `SemiDiscretizationMethod.jl` [25], which also contains the recent multiplication-free variant with linear time complexity [26]. The great strength of this family is generality: time-periodic coefficients (e.g., milling) are handled by the same machinery through Floquet theory. For *autonomous* systems, however, this generality is paid for at every single parameter point: a large discretization matrix is built and its dominant eigenvalues are computed, although only the sign of the rightmost characteristic exponent – one bit of information – decides stability. Moreover, delayed systems of neutral type (where the essential spectrum destroys the convergence of many discretizations [27, 28]), systems with algebraic constraints, i.e., delay differential–algebraic equations (DDAEs) [29, 30], distributed delays [31], and fractional-order systems [32, 33, 34] each require dedicated, nontrivial extensions in the time domain.

The frequency domain offers a complementary route, since all the listed system classes share the same abstract object: an analytic characteristic function $\mathcal{D}(\lambda)$ whose roots in the right half-plane must be counted (for a comprehensive treatment see [35]). Classical tools include the D-subdivision method [36], the analytical stability criteria of Stépán [6], the Nyquist and Mikhailov criteria [37, 38], the direct method of Walton and Marshall [39], the Rekasius substitution [40], the cluster treatment of characteristic roots [41], and the frequency-sweeping tests of Li, Niculescu and Çela [42]; the Lambert-W formalism [43] and the matrix method of Louisell [44] target the dominant roots directly, the quasi-polynomial root finder QPmR [45, 46] computes all roots in a prescribed rectangle from the values of $\mathcal{D}(\lambda)$ on a grid – a contour-integral strategy also developed for general nonlinear eigenvalue problems by Beyn [47] – and the YALTA toolbox [48] automates root-locus and stability analysis for classical and fractional delay systems. A recent survey of argument-principle-based counting techniques is given in [49].

Two frequency-domain strategies are particularly relevant here. The first solves the two real equations $\operatorname{Re}\mathcal{D}(\mathrm{i}\omega) = \operatorname{Im}\mathcal{D}(\mathrm{i}\omega) = 0$ in the parameter space extended by the frequency ω, which yields the D-curves, i.e., the parameter combinations where a root crosses the imaginary axis. The multi-dimensional bisection method (MDBM) [50, 51] traces these one-dimensional solution manifolds very efficiently. The D-curves, however, are *not* the stability bound-

aries: they merely mark root crossings, and the number of unstable roots on either side must be decided separately – by root-crossing directions from a test point per cell [6, 41], or, for turning, as the lower envelope of the lobes [2, 52]. Towards low spindle speeds the lobes become infinitely dense and an ever smaller fraction of the D-curve length bounds the stable region [53, 54]; for models with several parameters and delays most of the effort goes to curves that never bound it.

The second strategy counts the unstable roots directly by the argument principle, i.e., by the winding number of $\mathcal{D}(\mathrm{i}\omega)$ – the generalized Nyquist, or Mikhailov, criterion [6, 55]. Root counting by contour integrals of $\mathcal{D}'/\mathcal{D}$ or by the argument increment along sampled contours has a long history for general analytic functions [56, 57, 58, 59, 60], and for fractional delay systems an argument-principle stability test was given by Hwang and Cheng [61]; for delay systems, Hassard [62] derived a semi-analytical counting formula, Zítek and Vyhlídal formulated argument-increment criteria for neutral systems [63, 64], and, closest to the present work, Xu and Wang [65] and Xu, Stépán and Wang [66] evaluated the argument-principle "testing integral" numerically, observing that a rough quadrature suffices because the result must be an integer; Xu and Wang also gave a scheme for the rightmost roots built on that integral [65]. Outside the delay literature, winding numbers of analytic functions are computed routinely in the spectral stability analysis of travelling waves, where the Evans function is evaluated on an adaptively refined contour that keeps the relative change between consecutive points below a tolerance [67, 68]. The appeal of root counting is that the answer $Z(\boldsymbol{p})$ is exactly the quantity a stability chart needs. Its curse is twofold. First, $Z(\boldsymbol{p})$ is a piecewise-constant integer field: it jumps abruptly across the stability boundaries, so it cannot be interpolated, and brute-force charts need fine parameter grids. Second, the numerical evaluation is hardest *exactly at the stability boundary* – the only region one actually cares about – because as a root approaches the integration line the integrand of the winding integral develops a near-singular peak whose width shrinks to zero, and fixed grids or rough quadratures either miss the peak or need excessive refinement; the infinite frequency range poses an additional truncation problem.

This paper revisits the integration-based Nyquist criterion and turns it into a fast and robust tool with *interpolable* output. Its speed is best stated per thread: a parameter point of the benchmark examples costs 4–20 μs on one CPU thread, and this cost divides almost exactly among CPU threads and, on a graphics processor, in proportion to the number of threads up to the saturation of the device (Section 8). The contributions are the following. (i) The winding integral is reformulated as a scalar ordinary differential equation for the phase, marched by a high-order adaptive integrator [69, 70, 71] whose step-size control resolves the near-singular peaks and crosses the practically unbounded frequency range, with the integrand evaluated exactly by forward-mode automatic differentiation [72]. (ii) Since the count needs only the end value of the continuous phase, the integration can be dropped altogether: the exact phase increments between adaptively placed samples are summed (discrete phase unwrapping), and the exact phase slope from the same automatic derivatives drives an ODE-style step-size control that rejects steps hiding a root transition. This integration-free march needs one evaluation of $\mathcal{D}$ per step and some 30–130 evaluations for a median parameter point, is 38 to 298 times faster than the phase ODE on the three benchmarks of Section 7.1 and more robust near the boundary, and – consisting of identical scalar steps on a fixed-size state – maps directly onto GPUs,

where single precision suffices. Measured on two CPUs and four GPUs, its chart time is proportional to $1/n$ in the number of threads over several decades up to the saturation of the device, and a model with the single-thread cost and the effective parallelism of the device estimates it for other hardware. On a high-end GPU a full-HD chart is recomputed while a parameter slider moves; the same kernel runs in any web browser (WebGPU) and on rented cloud GPUs, where brute-force scans of four- to six-dimensional parameter spaces become a matter of minutes to hours. (iii) The location of the characteristic root closest to the integration line – normally the rightmost root – is estimated *during the same sweep* at negligible extra cost; the first-order estimate is the smooth default, whose sign is that of the rightmost root near the stability boundary, and an optional local polish (Taylor, Newton or a combined mode) sharpens it toward machine precision for a modest fraction of the cost when an accurate root is wanted. Tracking several local minima yields multiple roots at once. This ordered march is what distinguishes the phase ODE from plain quadrature, whose recursive subdivision visits the frequencies out of order, so that the minima could only be found afterwards from stored nodes. (iv) The root estimate makes the chart interpolable: a combined coloring shows the integer number of unstable roots outside and the smooth stability margin inside the stable domain, an effective visual diagnostic. (v) For the integration back-end, the integrality of the computed winding number provides a built-in, dimensionless error estimate that scales with the requested solver tolerance; where it is structurally blind – a skipped peak costs exactly $\pm\pi$ and leaves the result an integer – the tracked root and the parity of the real roots supply the missing checks, and the discrete march avoids the failure by sampling $\mathcal{D}$ itself. (vi) The estimated root distance serves as a scalar objective for MDBM, which then traces the *true* stability boundary (not the D-curves) at very high equivalent resolution, and the most robust parameter point is found as the center of the largest circle inscribed in the stable domain. (vii) The characteristic function is extracted algorithmically from a simulation-ready right-hand side – including singular mass matrices (DDAEs) – so the user never has to derive $\mathcal{D}(\lambda)$ by hand. (viii) The method applies verbatim to retarded, neutral, distributed-delay, transcendental, high-dimensional and (preliminarily) fractional-order systems, and is released as the open-source Julia [73] package `InterpolatedNyquist.jl`. Compared with the rough-quadrature testing integral of [65, 66], the present formulation adds adaptive error control tied to the integer-residual estimate, the rightmost-root by-product that enables interpolation and boundary tracing, and a far wider class of demonstrated applications; compared with contour-moment methods such as ZEAL [59], it needs no bounded search region and is tailored to the half-line stability sweep. Adaptive phase unwrapping itself is classical: the acceptance test of the discrete march – the observed phase increment must agree with the trapezoidal integral of the exact phase slope – is Tribolet's consistency check [74], and keeping the increments resolved by adaptive sampling is the device of Ying and Katz [57] and of the Evans-function codes [67, 68]. What the discrete march adds is a step-size controller that also *grows* the step (rather than only subdividing a fixed grid) on the semi-infinite frequency axis of a transcendental function, exact slopes of an arbitrary user function by automatic differentiation, the in-stream root estimate – obtained from the minima of $|\mathcal{D}|$ on the samples already taken, without further evaluations of $\mathcal{D}$ – the decision of sub-resolution transitions by the side of that root, and a fixed-state formulation that runs as a GPU kernel.

The paper is organized as follows. Section 2 summarizes the system classes and the argument-principle count-

ing formula. Section 3 presents the phase ODE, the rightmost-root tracking, the higher-order refinement, the self-validating error estimate, and the integration-free discrete phase unwrapping. Section 4 assembles these ingredients into fast stability charts. Section 5 describes the automatic extraction of $\mathcal{D}(\lambda)$ from a time-domain model. Section 6 walks through the complete workflow on a constrained two-degree-of-freedom system with delayed control, Section 7 quantifies accuracy, CPU time and memory, including a comparison with the semi-discretization method, Section 8 measures how the cost divides among CPU and GPU threads and what that means for real-time charts, browsers and cloud GPUs, and Section 9 concludes. Appendix A collects twelve further case studies from all supported system classes, Appendix B lists the discrete march in algorithmic form, and Appendix C collects the detailed measurements of the integration back-ends, of the tolerance and of narrow number formats.

2. Linear autonomous delay systems and the argument principle

2.1. System classes and the characteristic function

Consider the linear autonomous delay system in first-order descriptor form

$$\mathbf{E}\,\dot{\boldsymbol{x}}(t) = \mathbf{A}_0\,\boldsymbol{x}(t) + \sum_{j=1}^{m}\Big(\mathbf{A}_j\,\boldsymbol{x}(t-\tau_j) + \mathbf{B}_j\,\dot{\boldsymbol{x}}(t-\tau_j)\Big) + \int_0^{\vartheta_{\max}} \mathbf{W}(\vartheta)\,\boldsymbol{x}(t-\vartheta)\,\mathrm{d}\vartheta, \tag{1}$$

with state $\boldsymbol{x}(t) \in \mathbb{R}^N$, constant matrices $\mathbf{E}, \mathbf{A}_j, \mathbf{B}_j \in \mathbb{R}^{N\times N}$, discrete delays $\tau_j > 0$ and a distributed-delay kernel $\mathbf{W}$. The exponential ansatz $\boldsymbol{x}(t) = \boldsymbol{\xi}\,\mathrm{e}^{\lambda t}$ leads to the characteristic matrix and the characteristic function

$$\mathbf{M}(\lambda) = \lambda\mathbf{E} - \mathbf{A}_0 - \sum_{j=1}^{m}(\mathbf{A}_j + \lambda\mathbf{B}_j)\mathrm{e}^{-\lambda\tau_j} - \int_0^{\vartheta_{\max}} \mathbf{W}(\vartheta)\,\mathrm{e}^{-\lambda\vartheta}\mathrm{d}\vartheta, \qquad \mathcal{D}(\lambda) = \det\mathbf{M}(\lambda). \tag{2}$$

Equation (1) covers all the system classes treated in this paper: *retarded* DDEs ($\mathbf{E} = \mathbf{I}$, $\mathbf{B}_j = \mathbf{0}$), *neutral* DDEs ($\mathbf{B}_j \neq \mathbf{0}$), delay *differential–algebraic* equations (DDAEs, singular $\mathbf{E}$, e.g., due to kinematic constraints or algebraic force balances), and *distributed* delays. Beyond (1), the method below requires only an analytic function $\mathcal{D}(\lambda)$ that can be evaluated at complex arguments; hence transcendental characteristic functions of continua (e.g., cosh-type expressions of elastic rods [75]) and fractional-order quasi-polynomials containing terms λ^α, $\alpha \notin \mathbb{N}$, are admissible as well.

The equilibrium of (1) is asymptotically stable if all roots of $\mathcal{D}(\lambda) = 0$ satisfy $\mathrm{Re}\,\lambda < 0$. More generally, a prescribed asymptotic decay rate (stability margin) $|\sigma|$ – often called γ-stability – requires all roots to lie left of the shifted line $\mathrm{Re}\,\lambda = \sigma < 0$. We denote by $Z(\boldsymbol{p})$ the number of roots with $\mathrm{Re}\,\lambda > \sigma$ for the design parameters $\boldsymbol{p}$; the stability chart is the map $\boldsymbol{p} \mapsto Z(\boldsymbol{p})$, and the stable (or γ-stable) domain is the region $Z = 0$.

For large $|\lambda|$ the characteristic function grows like $\mathcal{D}(\lambda) \sim c\,\lambda^n$, where the leading order n equals N for retarded systems in state-space form but is smaller for DDAEs (the algebraic rows remove dynamic content); for neutral systems $\mathcal{D} \sim c\lambda^n(1 + \sum_j b_j\mathrm{e}^{-\lambda\tau_j})$ with an integer n but a bracket that does not tend to a constant, and for fractional systems n is in general not an integer. The counting formula below needs n, and Section 3.1 shows that it can be estimated numerically without any structural knowledge.

2.2. Counting unstable roots on a half-line

Let the integration line $\lambda = \sigma + \mathrm{i}\omega$ be free of characteristic roots, and let $\mathcal{D}$ be analytic with finitely many roots to the right of the line. Applying the argument principle to the D-shaped contour composed of the line and the arc at infinity, and exploiting the conjugate symmetry $\mathcal{D}(\overline{\lambda}) = \overline{\mathcal{D}(\lambda)}$ of real-coefficient systems, the number of roots to the right of the line is obtained from frequencies $\omega \geq 0$ only [6, 62]:

$$Z = \frac{n}{2} - \frac{1}{\pi}\Delta\theta, \qquad \Delta\theta = \int_0^\infty \theta'(\omega)\,\mathrm{d}\omega, \qquad \theta(\omega) = \arg \mathcal{D}(\sigma + \mathrm{i}\omega), \tag{3}$$

where θ is continuous in ω and $n/2$ accounts for the phase gathered on the arc at infinity, where $\mathcal{D} \sim c\lambda^n$. This is the half-line (Mikhailov-type) form of the Nyquist criterion. Writing $\mathcal{D}'(\omega) \equiv \partial_\omega \mathcal{D}(\sigma + \mathrm{i}\omega)$ for the derivative along the line, the phase slope is

$$\theta'(\omega) = \operatorname{Im} \frac{\mathcal{D}'(\omega)}{\mathcal{D}(\sigma + \mathrm{i}\omega)} = \frac{\operatorname{Re}\mathcal{D} \cdot \partial_\omega \operatorname{Im}\mathcal{D} - \operatorname{Im}\mathcal{D} \cdot \partial_\omega \operatorname{Re}\mathcal{D}}{|\mathcal{D}|^2}. \tag{4}$$

For $\sigma = 0$, Z counts the unstable roots; for $\sigma < 0$ it counts the roots violating the prescribed decay rate, so the same formula draws γ-stability charts.

Three standing assumptions deserve explicit statement. *(i) Finitely many roots right of the line, none on it.* What decides this is the asymptotic behavior of $\mathcal{D}$ along vertical lines, $\mathcal{D}(\lambda) \approx c\lambda^n p(\lambda)$ with an exponential polynomial p ($p \equiv 1$ for retarded systems): the count is finite if all zeros of p lie left of the line. For retarded systems this holds for every σ. A DDAE whose algebraic constraints carry no delay – such as the index-3 showcase of Section 6, whose constraint $x_3 - x_2 = 0$ is delay-free – keeps the retarded character, whereas delayed algebraic variables make it of neutral type, and the spectrum can then acquire advanced-type root chains that run into the right half-plane, so that the count is infinite for any line [76, 30] – such systems are outside the scope of (3), although the diagnostic of Section 3.4 flags them, because the truncated integral then fails to settle to an integer. *(ii) Neutral tail.* For neutral systems $p(\lambda) = 1 + \sum_j b_j \mathrm{e}^{-\lambda\tau_j}$ and its phase oscillates without decay as $\omega \to \infty$: the integral in (3) does not converge but oscillates within a bounded envelope. If the neutral amplitude on the line satisfies $\beta_\sigma = \sum_j |b_j| \mathrm{e}^{-\sigma\tau_j} < 1$ – for $\sigma = 0$ the strong-stability condition of the difference operator [27, 28] in the scalar case; for matrix coefficients $\mathbf{B}_j$ the corresponding condition is on the spectral radius of $\sum_j \mathbf{B}_j \mathrm{e}^{\mathrm{i}\vartheta_j}$ over all ϑ_j – then $|p - 1| \leq \beta_\sigma$ on the whole line, the phase of p stays within $\arcsin(\beta_\sigma)$ in total, the oscillation of the count within $\arcsin(\beta_\sigma)/\pi < 1/2$, and truncating at a generic $\omega_{\max}$ and rounding still recovers the exact count. As $\beta_\sigma \to 1$ (essential instability approaching) the bound saturates and the integer residual of Section 3.4 grows accordingly – the diagnostic degrades exactly where the answer itself becomes fragile (see the acceleration-feedback case study A.6 in Appendix A). A shifted line with $\beta_\sigma \geq 1$ lies left of the essential spectrum, and no count is defined there. *(iii) Analyticity right of the line.* Fractional powers λ^α are taken on the principal branch, whose cut lies on the negative real axis; for $\sigma \geq 0$ the region right of the line is then free of branch points, and (3) applies verbatim; the branch point at $\lambda = 0$ lies on the line, but its integrand singularity $\sim \omega^{\mu-1}$ is integrable for $\mu > 0$ provided $\mathcal{D}(0) \neq 0$ (otherwise the origin is a root on the line and no count is defined). A shifted line with $\sigma < 0$, however, encloses part of the cut, so the argument principle does not apply there; the fractional case studies of Appendix A show how the violation announces itself.

2.3. Why the naive evaluation is impractical

Formula (3) looks like a textbook quadrature problem, but three properties make its direct use for stability charts inefficient. First, $Z(\boldsymbol{p})$ is integer-valued: it is constant inside every cell of the chart and jumps at the boundaries, so the map cannot be interpolated, and any grid-based chart must be fine enough to resolve every cell of interest. Second, the numerical evaluation is hardest exactly where it matters most. If a root $\lambda_r = \sigma_r + \mathrm{i}\omega_r$ lies close to the integration line, the integrand (4) develops a near-Lorentzian peak at $\omega \approx \omega_r$,

$$\theta'(\omega) \approx \frac{-(\sigma_r - \sigma)}{(\sigma_r - \sigma)^2 + (\omega - \omega_r)^2} + \text{smooth part}, \tag{5}$$

whose half-width $|\sigma_r - \sigma|$ tends to zero as the parameter point approaches the stability boundary, while its integral contribution stays $\pm\pi$; *any* method that samples only the phase slope θ' and happens to place all its samples outside such a peak miscounts Z by exactly one, and – since the result stays an integer – does so without any warning. Third, the integration range is infinite: the phase slope decays only algebraically, with delay-induced oscillatory ripples, so a fixed truncation $\omega_{\max}$ must balance truncation error against the cost of resolving a ripple over a huge interval.

Section 3 addresses the first and third difficulties with the step-size control of an adaptive integrator, and turns the second into an asset by extracting the location of the offending root from the very peak that makes the integration hard. The peak-skipping hazard, by contrast, cannot be removed by any choice of quadrature of θ'; what can be done is to detect it – by the consistency of the count with the tracked root and with the parity of the real roots (Section 3.4) – or to avoid it by sampling $\mathcal{D}$ itself rather than its phase slope, which the discrete march of Section 3.6 does.

3. Ordered adaptive integration with rightmost-root tracking

3.1. The winding integral as an ordinary differential equation

The key step is simple: instead of applying quadrature to (3), the accumulated phase is treated as the state of the scalar initial value problem

$$y'(\omega) = \theta'(\omega) = \operatorname{Im} \frac{\mathcal{D}'(\omega)}{\mathcal{D}(\sigma + \mathrm{i}\omega)}, \qquad y(0) = 0, \qquad \tilde{Z} = \frac{n}{2} - \frac{y(\omega_{\max})}{\pi}, \qquad Z = \operatorname{round}(\tilde{Z}), \tag{6}$$

which is handed to a modern adaptive integrator with local error control. It should be stated plainly what this formulation is and is not. The right-hand side of (6) does not depend on the state y, so the Jacobian $\partial_y y'$ vanishes identically: (6) is a quadrature problem in disguise, it is never stiff in the classical sense, and an automatically stiffness-switching integrator provably never switches on it. What the integrator contributes is therefore not stiffness handling but *adaptive step-size control on an ordered march*: steps contract around the near-singular peaks (5), where the phase turns by $\pm\pi$ over a frequency interval of width $|\sigma_r - \sigma|$, and expand again over the smooth stretches. Because the integrand is analytic, a high-order explicit Runge–Kutta pair is the appropriate choice; the default is `Vern9` [69], which at the default tolerance is three orders of magnitude more accurate than the fifth-order alternative [70] for a factor of 2.4 in

cost (Section Appendix C.1). The ordered march is what makes the root tracking of Section 3.2 possible, which is the reason the phase ODE is preferred here over plain quadrature (Section 3.5).

The integrand is evaluated *exactly*: the characteristic function is called once per frequency with a dual-number argument $\lambda = \sigma + \mathrm{i}(\omega + \delta)$, $\delta^2 = 0$, so forward-mode automatic differentiation [72] delivers $\mathcal{D}$ and $\mathcal{D}' = \partial_\omega \mathcal{D}$ simultaneously, without finite differencing and without any user-supplied derivative. A tiny offset $\omega \mapsto \max(\omega, 10^{-9})$ regularizes branch points of fractional terms at the origin.

Two practical parameters remain. The truncation frequency $\omega_{\max}$ can be chosen generously: a *too small* $\omega_{\max}$ is visible in the result, because the truncated tail shifts $\tilde{Z}$ away from an integer (Section 3.4). The cost of the tail is set by the decay of the delay-induced phase ripple. If the delayed terms sit at least two orders below the leading power of $\mathcal{D}$ (delayed *position* feedback), the ripple decays at least like ω^{-2} and a wide window is essentially free: the fourth-order benchmark costs 1426, 1474 and 1506 evaluations at $\omega_{\max} = 10^6$, 10^8 and 10^{10}. If a delayed term sits only one order below (delayed *velocity* feedback), the ripple decays like ω^{-1}, the solver keeps resolving it (Fig. 2c), and $\omega_{\max}$ does cost. Hence: *for accuracy, be generous* ($\omega_{\max} = 10^6$ is safe for every system in this paper); *for speed, be frugal* – two to three decades above the highest resonance leave a tail of order $0.2/\omega_{\max}$ in $\tilde{Z}$, far below the rounding threshold.

The leading order n in (6) is estimated numerically, and the natural place to do so is not the imaginary axis at all. The term $n/2$ accounts for the phase gathered on the arc at infinity, and that arc lies in the *right half-plane*, where every delayed factor $\mathrm{e}^{-\lambda\tau_j}$ decays exponentially. Probing along the positive real axis therefore measures precisely the quantity the counting formula needs, with the delay terms already extinguished:

$$n \;\approx\; \underset{s_k \in [s_*/10,\, s_*]}{\text{LS-slope}} \Big(\ln s_k \;\mapsto\; \ln\big|\mathcal{D}(s_k)\big| \Big), \qquad s_* \gg 1, \tag{7}$$

the least-squares slope of the log–log magnitude over a handful of log-spaced real samples. The probe is decoupled from the integration range (n is a property of $\mathcal{D}$ alone; the default is $s_* = 10^8$ whatever $\omega_{\max}$ is), costs a handful of evaluations once per sweep, and converges as $O(1/s_*)$ – measured here as $7.5\cdot10^{-5}$, $7.4\cdot10^{-7}$ and $7.4\cdot10^{-9}$ at $s_* = 10^4$, 10^6 and 10^8 – including the effective integer order of neutral equations and the non-integer order of fractional ones. On the imaginary axis, by contrast, the delay terms superimpose a ripple on $\ln|\mathcal{D}(\mathrm{i}\omega)|$ that never decays for a neutral system (a single-point estimator gives $n_{\mathrm{est}} = 80\,028$ at $\omega = 10^6$ for a true effective order of 2). Two limitations remain. $|\mathcal{D}(s)| \sim s^n$ overflows for large n, so s_* is reduced automatically; the better answer is structural: where the model offers an open-loop/feedback split, the *return difference* $\mathcal{D} = \det(\mathbf{I} + \mathbf{Q}_0^{-1}\mathbf{F}) = 1 + L(\lambda)$ has the same right-half-plane zeros, adds poles only at the (stable) open-loop roots, and tends to 1 at infinity, so $n = 0$ *exactly* (case study A.9). And Eq. (7) presumes polynomial growth, which fails for continua such as $\cosh(\sqrt{\lambda^2 + c\lambda}\,L) \sim \mathrm{e}^{\lambda L}$; the package detects this (the slope is not reproduced one window lower) and fits along the imaginary axis instead ($n \approx 0$ for the rod of Appendix A).

The requested relative and absolute solver tolerances control the local error of the accumulated phase, measured in radians. Since the exact winding number is an integer multiple of π in the phase, the error of $\tilde{Z}$ inherits the tolerance directly as a *dimensionless* quantity: the same default (10^{-5}) is appropriate for a machining model and for a dense

50×50 matrix determinant alike, with no problem-dependent scaling (Section 3.4).

3.2. Rightmost-root estimation as a by-product

The peaks of the integrand are not only a numerical nuisance – they carry the information where the dominant characteristic roots are. Along the integration line, $|\mathcal{D}(\sigma + \mathrm{i}\omega)|^2$ dips to a sharp local minimum at $\omega \approx \omega_r$ for every nearby root $\lambda_r = \sigma_r + \mathrm{i}\omega_r$, and the smallest minimum belongs to the root closest to the line, which for $\sigma = 0$ is typically the rightmost root that decides stability. The solver therefore tracks $\min_\omega |\mathcal{D}|^2$ during the march at negligible cost: at every integrand evaluation the already computed quantities $\mathcal{D}$ and $\mathcal{D}'$ yield the real part of one Newton step towards the root,

$$\hat{\sigma} \;=\; \sigma + \mathrm{Re}\!\left(-\frac{\mathcal{D}}{\mathrm{d}\mathcal{D}/\mathrm{d}\lambda}\right) \;=\; \sigma - \frac{\mathrm{Re}\,\mathcal{D}\cdot\partial_\omega \,\mathrm{Im}\,\mathcal{D} - \mathrm{Im}\,\mathcal{D}\cdot\partial_\omega \,\mathrm{Re}\,\mathcal{D}}{|\mathcal{D}'|^2}, \tag{8}$$

using $\mathrm{d}\mathcal{D}/\mathrm{d}\lambda = -\mathrm{i}\,\mathcal{D}'$, and the frequency of the minimum is stored as $\hat{\omega}$. The pair $(\hat{\sigma}, \hat{\omega})$ is a first-order estimate of the dominant root location returned together with Z. Its accuracy is highest near the stability boundary, where the root lies close to the line, provided a sample lands close to the minimum: a Newton step from a sample at a distance $\delta\omega$ from ω_r carries an error of order $|\mathcal{D}_{\lambda\lambda}/\mathcal{D}_\lambda|\,\delta\omega^2$, so once the peak is far narrower than the local step, the estimate is limited by the step rather than by the distance of the root from the line (Table 5). Away from the boundary the estimate degrades gracefully, but there only its sign and magnitude matter – as a smooth measure of the distance to instability.

Several roots can be tracked simultaneously. A compile-time variant of the solver detects every local minimum of $|\mathcal{D}|^2$ through the sign change of its ω-derivative, $2(\mathrm{Re}\,\mathcal{D}\,\partial_\omega \,\mathrm{Re}\,\mathcal{D} + \mathrm{Im}\,\mathcal{D}\,\partial_\omega \,\mathrm{Im}\,\mathcal{D})$, and maintains the N_r best candidates in a sorted, allocation-free buffer; a boundary test at $\omega = 0$ captures real dominant roots, since $|\mathcal{D}|^2$ is even in ω. Tracking one root is the default and adds a few percent to the sweep time; tracking zero roots gives maximal sweep speed. One caveat deserves emphasis: the *deepest* minimum of $|\mathcal{D}|$ belongs to the rightmost root near the stability boundary, but deeper inside the stable or unstable domains a different root branch may produce a deeper dip. When the true dominant root is required away from the boundary, several minima should be tracked ($N_r \approx 5$–10), all of them refined, and the dominant root taken as the one with the maximal real part – this recipe is validated against an independent eigenvalue method in Section 7.3.

The recommendation matters because the single-minimum estimate is not only inaccurate away from the boundary but *discontinuous*: where one branch overtakes another as the closest one, $\hat{\sigma}$ jumps, and a tracer that fits a local linear model (Section 4.2) reads the jump as a spurious zero crossing. The dominant root, a maximum over branches, passes through such crossings continuously, so the coloring and the boundary objective are built from it.

3.3. Optional higher-order root refinement

The raw tracked estimate $(\hat{\sigma}, \hat{\omega})$ – one Newton step off the minimum of $|\mathcal{D}|$ – is the method's *default* root output, and deliberately so. It is first-order accurate, but it is the *smoothest* field over a parameter chart (a single formula evaluated at every pixel), it is the cheapest, and near the stability boundary, where the classification is decided, its sign is that of the rightmost root (away from it, the closest minimum may belong to a non-dominant branch, Section 3.2).

When an accurate rightmost root is wanted, the same automatic derivatives drive an optional local polish. Writing the Taylor expansion at $\hat{\lambda} = \hat{\sigma} + \mathrm{i}\hat{\omega}$,

$$\mathcal{D}(\hat{\lambda} + \Delta) \approx \mathcal{D} + \mathcal{D}_\lambda \Delta + \tfrac{1}{2}\mathcal{D}_{\lambda\lambda} \Delta^2 + \tfrac{1}{6}\mathcal{D}_{\lambda\lambda\lambda} \Delta^3 = 0, \tag{9}$$

where the subscripts denote derivatives with respect to λ evaluated at $\hat{\lambda}$, the package offers (i) *Newton–Raphson* iterations $\Delta = -\mathcal{D}/\mathcal{D}_\lambda$; (ii) a single *quadratic* (Poly-2) or (iii) *cubic* (Poly-3) Taylor solve, where the root of the local polynomial closest to $\hat{\lambda}$ is selected via the eigenvalues of its companion matrix; and (iv) a *combined* mode that runs the polynomial and Newton polishes and keeps, per root, the candidate with the smallest $|\mathcal{D}|$. The first derivative is exact (automatic differentiation); the higher derivatives are central differences of that exact first derivative, which avoids the compilation cost of nested dual numbers. Each polish is *monotone*: it is accepted only if it lowers $|\mathcal{D}|$ below the seed, otherwise the raw estimate is returned, so refinement never increases $|\mathcal{D}|$ – although, as discussed next, a lower $|\mathcal{D}|$ can belong to a neighboring root.

Which polish is best is system-dependent: on a smooth quasi-polynomial Newton reaches machine precision, but on a characteristic function with poles a Newton step can settle on a neighboring root, where the Taylor polishes stay put; the combined mode is the robust choice at about 25 % over the raw solve (Section 6.4). Refinement is therefore off for charts and on when a specific rightmost root matters; a precise decay-rate *map* comes from the σ-contours of Section 4.4. Applied to several tracked estimates, the refinement returns a set of dominant roots, as a dedicated root finder such as QPmR [45] does, here as an optional post-processing step of the count.

3.4. Self-validating error estimate

The exact winding number is an integer, so

$$\varepsilon = \left|\tilde{Z} - \mathrm{round}(\tilde{Z})\right| \tag{10}$$

is a freely available a-posteriori indicator of the accumulated error – the local integration errors, the tail truncated at $\omega_{\max}$ and the residual of the estimated order (7) – and a necessary condition, not a certificate. (A root *on* the line has no count; the package returns an invalid-count marker.) Since the phase is integrated in radians, ε is dimensionless and scales with the requested tolerance, amplified by the accepted-step count: the largest residual of a chart follows $\max \varepsilon \approx 100\,\mathrm{tol}$ (Section 6.3), so the tolerance is a universal accuracy dial that transfers between systems without re-tuning; the package default 10^{-5} keeps it two orders of magnitude below the rounding threshold. Whenever the leading order n is known by inspection, it should simply be passed (`n_power_max`); the estimator (7) only spares the user who does not know it.

The diagnostic presumes a *converging tail*. For retarded systems it does: the median residual of the phase ODE on the retarded case studies of Appendix A is $\sim 10^{-4}$ at the chart tolerance 10^{-4}. For neutral systems the tail oscillates within $\arcsin(\beta)/\pi$ (Section 2.2) and the residual inherits it (0.07–0.15 on the neutral panels, with correct counts); there the analytic bound $\arcsin(\beta)/\pi < 1/2$ protects the count, not ε.

One blind spot is inherited from the argument principle itself. A march that steps *over* a peak loses exactly $\pm\pi$ of phase, so $\tilde{Z}$ shifts by exactly one and remains an integer: ε measures the quadrature and cannot see a skipped peak. Near a stability boundary the peak half-width $|\sigma_r - \sigma|$ does collapse below any practical step (Table 5), and no integration back-end is immune; the discrete march of Section 3.6, which samples $\mathcal{D}$ itself, rejects such steps by construction (Table 4). Two free tests detect a skipped peak. The first compares the count with the tracked root of Section 3.2, which shares the samples but not the accumulation; restricted to the neighborhood of the line – far from it the tracked minimum need not be dominant, and a blind test fires on 459 of the 8000 points of a fully converged chart – the test

$$\operatorname{sign}(\hat{\sigma} - \sigma) \text{ disagrees with } [Z = 0] \qquad \text{and} \qquad |\hat{\sigma} - \sigma| < h \tag{11}$$

(h a small fraction of the spread of $\hat{\sigma}$, here $h = 0.001$) fires on only 2 of them, and it is sharp exactly where a skip flips the stable/unstable classification. The second is parity: complex roots right of the line come in conjugate pairs, so

$$Z \equiv \left[\operatorname{sign} \mathcal{D}(\sigma) \neq \operatorname{sign} c\right] \pmod 2, \tag{12}$$

with $\mathcal{D}(\sigma)$ and the sign of the leading coefficient c known from the first sample and the real-axis probe (7). A single skipped peak violates (12); in a repeated boundary stress test it caught all 13 miscounts of the ODE march and fired on none of the correct counts, whereas the sign test (11) missed 5 of them (`data/unwrap_checks`). Neither test rescues a chart run far outside the rule of thumb (at tol = 10^{-2} the near-boundary test catches only 10 of 89 flips): that failure is announced by the residual field lighting up over the whole chart. The minimum of $|\mathcal{D}|^2$ also *locates* a skipped peak however narrow it is, which an optional repair callback uses (Appendix C).

3.5. Alternative integrators within the same framework

The phase integrand (4) can also be fed to adaptive Gauss–Kronrod quadrature (`QuadGK.jl` [77]) through the identical interface; for the count alone it is competitive, within a factor of about three in evaluations either way (Table C.14). It visits the frequencies out of order, however, so no local minimum of $|\mathcal{D}|^2$ can be recognized on the fly – and every chart in this paper is built from the tracked root: the coloring (16), the boundary objective (17), the refinement seeds and the cross-check (11). This ordered traversal, not accuracy or stiffness, is why the phase ODE is the default integration back-end. A fixed-step trapezoidal back-end is included as a real-time floor, but a uniform grid cannot resolve the peak (5) of a root close to the line: its mean count error converges, its worst error over a chart does not (Appendix C). The fourth back-end keeps the adaptivity and the ordered traversal but drops the integration itself.

3.6. Discrete phase unwrapping: counting without integration

The phase ODE (6) integrates θ' only to obtain one number, the end value $y(\omega_{\max}) = \theta(\omega_{\max}) - \theta(0)$ of the continuous phase $\theta(\omega) = \arg \mathcal{D}(\sigma + \mathrm{i}\omega)$. That number is available *without* integration. Between two samples $\omega_a < \omega_b$ the phase increment is known exactly up to a multiple of 2π,

$$\Delta_{ab} = \operatorname{Arg} \frac{\mathcal{D}(\sigma + \mathrm{i}\omega_b)}{\mathcal{D}(\sigma + \mathrm{i}\omega_a)} \in (-\pi, \pi], \tag{13}$$

and it equals the true increment $\Phi(\omega_b) - \Phi(\omega_a)$ whenever the latter lies in $(-\pi, \pi)$. On any sampling $0 = \omega_0 < \omega_1 < \cdots < \omega_K = \omega_{\max}$ that is dense enough in this sense, the counting formula (3) becomes the finite sum

$$\tilde{Z} = \frac{n}{2} - \frac{1}{\pi} \sum_{k=1}^{K} \Delta_{k-1,k}, \tag{14}$$

which is *exact*: there is no integration error, only the requirement that no branch of the phase is lost – the classical phase-unwrapping problem (Tribolet [74]; Ying and Katz [57], with a rigorous variant in [60]; for delay systems Stépán [6] and Hassard [62]; cf. Section 1 for what is new here). The same dual-number evaluation that returns $\mathcal{D}$ also returns the exact phase slope $\theta' = \mathrm{Im}(\mathcal{D}'/\mathcal{D})$ at every sample.

Adaptivity.. The slope makes the march adaptive in exactly the way an embedded Runge–Kutta pair is. A trial step $h = \omega_b - \omega_a$ evaluates $\mathcal{D}$ and $\mathcal{D}'$ at ω_b and compares the exact (observed) increment with the trapezoidal value predicted by the exact slopes at both ends,

$$e = \left| \Delta_{ab} - \tfrac{h}{2}(\theta'_a + \theta'_b) \right|, \tag{15}$$

accepts the step if $e \le$ tol (in radians; default 0.3), and sets the next trial step to $h \cdot \min\{4, \max\{0.2,\ 0.9(\mathrm{tol}/e)^{1/3}\}\}$, the exponent reflecting the $O(h^3\theta''')$ trapezoid error – the standard step-size controller, with the exact increment in the role of the higher-order solution. The acceptance test is Tribolet's consistency check; the controller, which also lets the step grow, is what crosses the semi-infinite frequency axis in few steps. Unlike the solver tolerance of Section 3.1, this tolerance does not set the error of $\tilde{Z}$ – the sum (14) has none – but is a safety margin against losing a branch, and it is far from critical (Appendix B). On smooth stretches e follows the curvature of the phase. Across an unresolved root transition – a peak (5) of width $|\sigma_r - \sigma|$ hidden inside the step – the observed increment contains a jump of almost $\pm\pi$ that the end-point slopes do not predict, $e \approx \pi$, and the step is rejected and refined until the transition is resolved. In the tail of a retarded system, where θ approaches its limit up to a decaying ripple, $e \to 0$ and the step grows geometrically, so the frequency axis is crossed in a logarithmic number of steps whatever $\omega_{\max}$ is. (For a neutral system the ripple does not decay, the step stays near $\mathrm{tol}/(\beta\tau)$ and the cost grows linearly with $\omega_{\max}$, so a neutral chart needs an explicit, moderate $\omega_{\max}$: with the windows of Appendix A, 200–500, the march reproduces every defined count of the three neutral panels at a median of 260–650 evaluations per point.) The delay-induced ripple that keeps an integrator busy for thousands of steps (Section 3.1) becomes invisible once its phase amplitude drops below the tolerance, because its contribution is contained *exactly* in the end-point phase. Each step costs a single evaluation of $\mathcal{D}$ (a `Vern9` step costs sixteen), and on the systems of this paper a median point needs 30–130 evaluations instead of 10^3–10^4 (Section 7.1). Automatic differentiation is still essential: the error estimate (15) needs the exact slope, and the root tracking below needs $\mathcal{D}'$.

Root tracking without extra evaluations.. The rightmost-root by-product of Section 3.2 carries over. A minimum of $|\mathcal{D}|$ inside an accepted step is recognized by the sign change of $\partial_\omega|\mathcal{D}|$ between its end points; it is located on the cubic Hermite interpolant of the complex function $\mathcal{D}(\omega)$ built from $\mathcal{D}$ and $\mathcal{D}'$ at both ends – $\mathcal{D}$ itself is smooth even

where its phase is not – and one Newton step (8) from the located minimum gives $(\hat{\sigma}, \hat{\omega})$; the estimate is kept only if the Newton step stays within the step that contains the minimum. No additional evaluation of $\mathcal{D}$ is spent. The minimum is detected through the sign change of $\partial_\omega|\mathcal{D}|$ between the end points, so a step that contains a minimum and a neighboring maximum of $|\mathcal{D}|$ hides the minimum; for a dominant root near the line the step is small and this does not happen.

Below the resolution of the number format.. When a root lies so close to the line that its transition is narrower than a floor of a few machine epsilons of the frequency ($h \le 8\,\epsilon_{\mathrm{mach}}\max(\omega, 1)$ in the format in which the march runs), the step is not refined further and the observed increment is ambiguous near $\pm\pi$. The branch is then decided by the *side* of the root: a root right of the line lowers the phase by π, one left of it raises it, and the side is the sign of the Newton estimate from the current sample – the peak repair of Section 6.3 at no cost. Such a count is a decision rather than a measurement, and the point is flagged. Since the sum (14) carries no integration error, the residual ε of (10) now contains only the truncated tail and the error of the order n: for retarded systems it is typically 10^{-7}–10^{-10}, and a value above $1/4$ flags a real root *on* the line ($\mathcal{D}(\sigma) = 0$, $\tilde{Z} = k + \frac{1}{2}$). A complex pair exactly on the line is not flagged – the count is then undefined, and the march returns one of the adjacent values – and for a neutral system the residual carries the bounded oscillation of Section 2.2, which exceeds $1/4$ for a neutral amplitude above about 0.7, so that the flag is not usable for that class.

Limitations.. An end-point test cannot see an increment of exactly 2π: two transitions in the same direction inside one step escape it. Two situations produced them on the benchmarks. First, a $\mathcal{D}$ with poles – a return difference $1 + L(\lambda)$ – places a pole next to every lightly damped mode; when a root leaves that mode into the right half-plane, the pole–zero pair winds the phase by -2π within a frequency interval of the size of the modal damping. Writing $\mathcal{D}$ as an entire function – multiplying out the (stable) denominators, which does not change the count in $\mathrm{Re}\,\lambda > \sigma$ – removes the pair. This reverses the advice of Section 3.1 for the return difference, whose $n = 0$ and moderate magnitudes favor the phase ODE; for large models with a low-rank feedback a step cap near the known open-loop poles would keep both advantages. Second, a regenerative delay τ can place a chain of roots near the axis, spaced $\approx 2\pi/\tau$ apart in frequency; capping the step at $h \le \pi/(2\tau_{\max})$ over the resonance band removes the ambiguity at a moderate cost (for the turning model of Table 3, $h \le 0.05$ for $\omega < 5$). A third situation is generic though it did not occur on the benchmarks: two lightly damped roots right of the line within one step, as near a double-Hopf point of a multi-mode system; the band cap covers it only if it is below their frequency separation. The step test is therefore a strong safeguard rather than a proof: on the 50×50 determinant of Appendix A (A.10), with fifty lightly damped modes packed into one decade of frequency, no setting removed all miscounts. With these rules the discrete march reproduces the reference counts on every point of the three benchmark charts and of nine gallery panels and the showcase (Appendix A), apart from points with a root exactly on the line, and near a Hopf boundary it is more robust than the phase ODE (Table 4). Where a parameter-independent, stable denominator cannot be cleared conveniently – $\cosh\gamma$ of a continuum, $\det \mathbf{Q}_0$ of a finite

element model – its phase can be measured once along the line and used as an effective order, $n_{\text{eff}} = 2\Phi_{\text{den}}(\omega_{\max})/\pi$, while the march counts the entire numerator (A.8, A.9).

Parallel hardware.. The march is a loop of identical scalar steps on a fixed-size state, one evaluation each, with no allocation; its execution on CPUs and GPUs is analyzed in Section 8, and the algorithm and its parameters are listed in Appendix B.

4. Fast stability charts

4.1. Brute-force sweep with interpolable coloring

The single-point solve of Section 3 is embarrassingly parallel over the parameter plane; a multithreaded sweep over a grid $\boldsymbol{p}_{ij}$ returns the fields Z, $\tilde{Z}$, $\min|\mathcal{D}|$, $\hat{\sigma}$ and $\hat{\omega}$. The integer field Z alone is a poor plotting quantity – flat plateaus separated by jumps – but the estimated root location repairs precisely this deficiency. The combined field

$$C(\boldsymbol{p}) = \begin{cases} \hat{\sigma}(\boldsymbol{p}), & \text{if } Z(\boldsymbol{p}) = 0, \\ Z(\boldsymbol{p}), & \text{otherwise,} \end{cases} \tag{16}$$

is a simple and effective coloring for visual inspection: inside the stable domain ($Z = 0$, $\hat{\sigma} < 0$) it varies *smoothly* and displays the stability margin, i.e., how far the dominant root lies from the imaginary axis, while outside it shows the integer number of unstable roots as discrete color levels. One glance at a single map thus reveals where the system is stable, how robustly stable it is there, and how many roots must be stabilized elsewhere. Because C is smooth over the stable domain, interpolation between grid points is meaningful exactly where it is useful, so a coarse and cheap grid suffices for the colored background; only the integer levels of the unstable side are plotted as flat colors.

4.2. High-resolution boundary tracing with MDBM

The stability boundary itself deserves better than pixel resolution. The multi-dimensional bisection method (MDBM) [50, 51] locates the zero set of a scalar function over a coarse initial mesh and refines only the bracketing cells, halving the mesh size in every iteration. Here *only the estimated root distance is used* as the scalar objective:

$$g(\boldsymbol{p}) = \operatorname{sign}\left([Z(\boldsymbol{p}) = 0] - \tfrac{1}{2}\right)\left|\hat{\sigma}(\boldsymbol{p}) - \sigma\right|, \tag{17}$$

which is positive inside the stable (γ-stable) domain, negative outside, crosses zero exactly on the stability boundary, and – unlike Z itself – varies almost linearly across it, because near the boundary $\hat{\sigma}$ is an accurate estimate of the dominant root's real part (Section 3.2). MDBM can therefore interpolate the zero crossing within each bracketing cell to sub-cell accuracy. One detail is essential rather than incidental: $\hat{\sigma}$ here must be the *dominant* root of Section 3.2, not the single closest minimum – the latter is discontinuous at branch crossings, and a linear model fitted inside a cell straddling the jump interpolates a phantom boundary point in the interior of the stable domain, corrupting anything built on the point cloud, such as the inscribed circle of Section 4.3. Note the contrast with applying MDBM to the

pair $\operatorname{Re}\mathcal{D} = \operatorname{Im}\mathcal{D} = 0$ in the parameter–frequency space: that approach traces *all* root-crossing D-curves, most of which lie deep in the unstable region (Section 1), while the objective (17) follows only the *true* stability limit. By the very nature of the bisection refinement, the computational effort concentrates exclusively on the boundary: k refinement levels raise the equivalent resolution of an $n_0 \times n_0$ initial mesh to $2^k n_0 \times 2^k n_0$, while the number of function evaluations grows only in proportion to the boundary length. On the fourth-order benchmark (Appendix A, case A.1), four refinement levels from a 20×20 mesh trace the boundary at a 320×320-equivalent resolution from under 2000 evaluations (Table 6) – some 50 times fewer than the 10^5 a brute-force grid of that density would need.

4.3. The most robust parameter point

With a high-resolution boundary and a smooth interior metric available, the natural design question – *which parameter point is the most robustly stable?* – reduces to a geometric one: the center of the largest circle inscribed in the stable domain maximizes the distance to the stability boundary in parameter space. The package computes it from the MDBM boundary point cloud (or from the binary stable mask of the brute-force grid) by a coarse-to-fine search followed by an omnidirectional pattern refinement with adaptive step control. Anisotropic parameter scaling turns the circle into an ellipse, which avoids degenerate results in elongated stable regions and lets the user weight the expected perturbations of the individual parameters. The exact radius is rarely of engineering interest; the *location* of the optimum is, and that location is insensitive to the residual discretization of the boundary. If a spectral-gap-optimal (rather than parameter-robust) point is preferred, the interpolable interior field $\hat{\sigma}(\boldsymbol{p})$ of (16) can be minimized instead – both options are cheap post-processing steps on data already computed.

4.4. σ-contours: decay-rate maps

Repeating the boundary tracing of Section 4.2 for a family of shifted lines $\sigma \in \{0, \sigma_1, \sigma_2, \dots\}$ produces the level curves of the asymptotic decay rate, i.e., a γ-stability map: each contour bounds the parameter region whose rightmost root lies left of $\operatorname{Re}\lambda = \sigma$. The interior coloring by $\hat{\sigma}$ and the traced σ-contours are two views of the same robustness landscape – the former is a fast estimate on a coarse grid, the latter a high-resolution traced boundary – and plotting them together provides a strong visual cross-validation of both (Section 6).

4.5. Recommended workflow

The complete chart-generation recipe used throughout the paper is:

1. *Colored sweep.* Multithreaded evaluation on a grid – with the discrete march of Section 3.6 for an entire $\mathcal{D}$, with the phase ODE (6) otherwise; plot the combined field (16). Cost: tens of milliseconds for a 100×100 grid with the discrete march (on a GPU, milliseconds for a full-HD grid), up to a few seconds with the phase ODE.
2. *Boundary tracing.* MDBM on the scalar objective (17), typically 4 refinement levels from a 20×20–30×30 initial mesh. Cost: comparable to or below the sweep.
3. *Robust point.* Largest inscribed circle/ellipse from the MDBM point cloud. Cost: milliseconds.

4. *Optional.* σ-contour family; root refinement at selected points; higher tolerance for publication-quality boundaries.

Every step reuses the same single-point solver, so the entire chart refreshes fast enough for interactive parameter exploration (Sections 7.1 and 8).

5. Automatic extraction of the characteristic function from a simulation model

In engineering practice, a system studied for stability almost always exists as a *simulation model* first: a right-hand side function written for a general-purpose integrator such as `DifferentialEquations.jl` [71]. Deriving the characteristic function (2) from it by hand is tedious and error-prone, particularly for many degrees of freedom, several delays, or algebraic constraints. The package therefore extracts $\mathcal{D}(\lambda)$ *algorithmically* from the very same right-hand side $\boldsymbol{f}(\boldsymbol{u}, \boldsymbol{h}, \boldsymbol{p}, t)$ used for time-domain simulation, where $\boldsymbol{h}(\boldsymbol{p}, t-\tau)$ is the standard history-function interface for the delayed states.

The mechanism requires a single evaluation of $\boldsymbol{f}$ per requested λ. The state is seeded with dual-number perturbations, $\boldsymbol{u} = \boldsymbol{u}_{\mathrm{eq}} + \boldsymbol{a}$, where the components of $\boldsymbol{a}$ carry independent infinitesimal directions, and the history argument is replaced by the *mock exponential history*

$$\boldsymbol{h}(\boldsymbol{p}, t) = \boldsymbol{u}_{\mathrm{eq}} + \boldsymbol{a}\, \mathrm{e}^{\lambda t}. \tag{18}$$

Whatever delayed value $\boldsymbol{h}(\boldsymbol{p}, t-\tau_j)$ the model requests contributes the factor $\mathrm{e}^{-\lambda\tau_j}$ automatically, so discrete, multiple, and (approximated) distributed delays are all captured without declaring them; delayed *derivatives*, i.e. neutral right-hand sides, are not yet supported by the extraction. Evaluating $\boldsymbol{f}$ at $t = 0$ and collecting the dual partial derivatives of the residual

$$\boldsymbol{r} = \lambda\, \mathbf{E}\, \boldsymbol{a} - \boldsymbol{f}(\boldsymbol{u}_{\mathrm{eq}} + \boldsymbol{a},\ \boldsymbol{h},\ \boldsymbol{p},\ 0) \qquad \Longrightarrow \qquad \mathbf{M}(\lambda) = \frac{\partial \boldsymbol{r}}{\partial \boldsymbol{a}}, \quad \mathcal{D}(\lambda) = \det \mathbf{M}(\lambda), \tag{19}$$

yields the exact characteristic matrix of the *linearized* dynamics around $\boldsymbol{u}_{\mathrm{eq}}$: the dual seeding performs the linearization, so the right-hand side may be fully *nonlinear* – no separate linearization step is required (the equilibrium itself could also be located from the same right-hand side, which is outside the scope of this paper; a residual check warns if the supplied $\boldsymbol{u}_{\mathrm{eq}}$ is not an equilibrium). The mass matrix $\mathbf{E}$ is supplied separately – it is not part of the right-hand side, exactly as in the `ODEFunction`/`DDEFunction` convention of `DifferentialEquations.jl` – and it may be *singular*: rows of zeros in $\mathbf{E}$ turn the corresponding rows of $\mathbf{M}(\lambda)$ into the algebraic constraint equations, so delay differential–algebraic systems are handled by the identical code path. The determinant is evaluated with statically sized arrays, so a full extraction costs one right-hand-side call plus an $N \times N$ determinant, and the dual seeding composes transparently with the outer dual numbers of the phase ODE (6).

The complete user workflow is thus reduced to providing the simulation right-hand side, the mass matrix if it differs from the identity, and the parameter ranges of the chart:

```
function rhs(u, h, p, t)              # simulation model (DDE interface)
```

Table 1: The price of never deriving $\mathcal{D}$ by hand. Both forms are the *same* function – they agree to machine precision and every one of the 10^4 counts is identical – and differ only in how each evaluation is obtained. Same system, same chart, same settings ($\omega_{\max} = 10^4$, default tolerance).

form of $\mathcal{D}$	chart (100×100)	per point [μs]
hand-derived (no determinant, no AD)	1.97 s	197
automatic extraction from the RHS	41.47 s	4147
ratio	21×	21×

```
    P, D = p
    x1_del = h(p, t - 0.5; idxs=1)    # delayed position
    v1_del = h(p, t - 0.5; idxs=4)    # delayed velocity
    ...                               # equations of motion + constraints
end

D(lam, p)  = get_D_from_model(rhs, lam, p, Val(7); mass_matrix = E)
                                      # Val(7): the model has 7 states;
                                      # E is the (singular) mass matrix
Z, Zraw, minD, sigma_est, omega_est =
    calculate_unstable_roots_direct(D, (2.0, 1.0))
```

The leading order n required by the counting formula (3) is estimated automatically by (7) – including the effective orders of neutral characteristic functions and the non-integer orders of fractional ones – so no additional structural input is needed. In Section 6 the extraction is verified against a hand-derived characteristic function: the two agree to machine precision.

5.1. What the convenience costs

Automatic extraction is a convenience, and it is not free. Every evaluation of the extracted $\mathcal{D}$ runs the user's right-hand side once with dual-number arguments and then takes a 7×7 determinant of the descriptor matrix, where the hand-derived form of the same system evaluates a flat scalar expression – a handful of multiplications, one exponential, no determinant and no automatic differentiation. Table 1 measures the difference on the identical chart: 1.97 s against 41.47 s for 100×100 points, a factor of about 21.

The trade is explicit, and both sides of it are defensible. If the model exists only as a simulation right-hand side – the normal situation early in a design study – the extracted form gives a correct chart in a minute with no derivation whatsoever. If the same chart is to be refreshed interactively or recomputed inside an optimization loop, the algebra is worth doing once: the hand-derived form makes the same chart about 21 times faster (1.97 *s* instead of 41.47 *s*). The recommended workflow is exactly that order – explore with the extracted form, derive $\mathcal{D}$ by hand once the model has

settled and speed starts to matter. All figures in this paper use the hand-derived form, having first verified it against the extracted one (Section 6).

6. Showcase: a constrained two-degree-of-freedom system with delayed PD control

All elements of the method are now demonstrated on a single mechanical example that is simple enough to verify by hand yet exercises every nontrivial feature at once: multiple degrees of freedom, a feedback delay, and an algebraic constraint, i.e., a delay differential–algebraic system with a singular mass matrix.

6.1. Model and automatic extraction

The main structure of mass m_1 is statically unstable (inverted-pendulum-like, modeled by the negative net suspension stiffness $k_1 < 0$ and damping c_1); it carries an absorber unit of mass m_2 attached through a spring and damper (k_2, c_2), and an instrumentation block of mass m_3 is *rigidly bolted* to the absorber mass. Instead of eliminating the rigid connection by hand, the model keeps it as an algebraic constraint with the internal force F_c – exactly the form in which a simulation model of a larger assembly would be written. A controller with a feedback delay (sampling is neglected) measures the motion of the main mass and applies the delayed PD force $u(t) = -P\,x_1(t-\tau) - D\,\dot{x}_1(t-\tau)$ to it. With the state vector $z = (x_1, x_2, x_3, v_1, v_2, v_3, F_\mathrm{c})$, the descriptor form (1) reads

$$\underbrace{\mathrm{diag}(1,1,1,m_1,m_2,m_3,0)}_{\mathbf{E}}\,\dot{z}(t) = \begin{pmatrix} v_1 \\ v_2 \\ v_3 \\ -k_1x_1 - c_1v_1 + k_2(x_2-x_1) + c_2(v_2-v_1) + u(t) \\ -k_2(x_2-x_1) - c_2(v_2-v_1) + F_\mathrm{c} \\ -F_\mathrm{c} \\ x_3 - x_2 \end{pmatrix}, \tag{20}$$

with the numerical values $m_1 = 1$, $m_2 = 0.3$, $m_3 = 0.2$, $k_1 = -1$, $k_2 = 1$, $c_1 = c_2 = 0.05$ and $\tau = 0.5$. The last row of (20) carries no time derivative – the mass matrix is singular – and the model is passed to the extraction of Section 5 exactly as written, in its simulation (right-hand-side) form. For verification, the constraint is also eliminated by hand ($x_3 = x_2$, merged mass $m_{23} = m_2 + m_3$), which yields the reduced quasi-polynomial

$$\mathcal{D}_\mathrm{red}(\lambda) = \det\begin{pmatrix} m_1\lambda^2 + (c_1{+}c_2)\lambda + k_1{+}k_2 + (P{+}D\lambda)\mathrm{e}^{-\lambda\tau} & -(c_2\lambda + k_2) \\ -(c_2\lambda + k_2) & m_{23}\lambda^2 + c_2\lambda + k_2 \end{pmatrix}. \tag{21}$$

The ratio of the automatically extracted 7×7 determinant to $\mathcal{D}_\mathrm{red}$ is constant in λ to machine precision (deviation below 10^{-15} in the computed charts), which validates the mass-matrix extraction path end to end; the estimated leading order (7) is $n \approx 4.000$, as expected for two mechanical degrees of freedom. Having verified the two forms identical, the results below use the cheaper hand-derived form (21); what the automatic extraction costs instead is measured in Section 5.1.

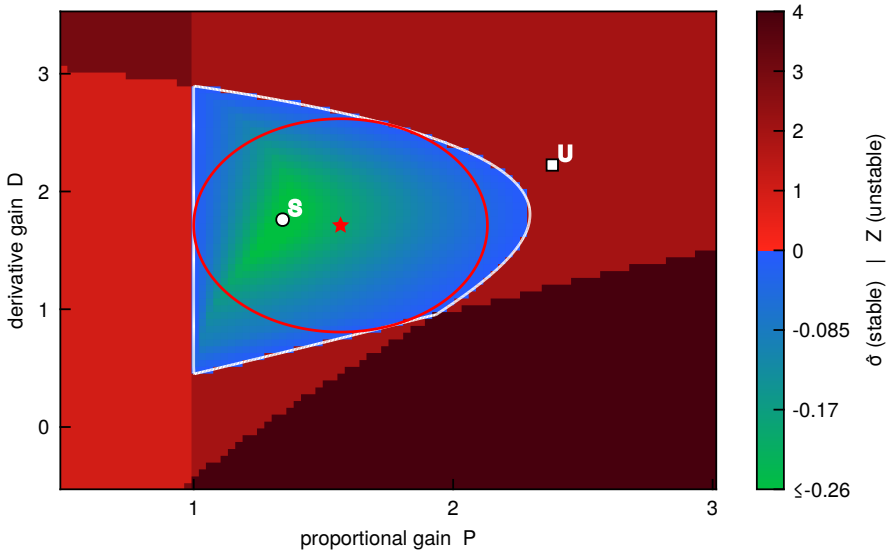


Figure 1: Stability chart of the constrained two-degree-of-freedom system with delayed PD control. Colored background: combined field (16) of the brute-force sweep (smooth stability margin $\hat{\sigma}$ inside the stable domain, integer number of unstable roots outside), computed with the discrete march of Section 3.6: 18 ms for the 90×70 grid on an Intel i5-10400 (twelve threads), against 1.5 s with the phase ODE, with identical counts apart from one point that the phase ODE miscounts (`data/unwrap_check_s01.csv`). Black curve: stability boundary traced by MDBM (phase ODE) with the scalar objective (17). Red ellipse and star: largest inscribed ellipse and the most robust parameter point. The markers S and U denote the stable and unstable points analyzed in Fig. 2.

6.2. *The chart and the phase integration at work*

Figure 1 shows the resulting stability chart over the control-gain plane (P, D). Since the uncontrolled structure is unstable, the stable domain is a bounded island: its left boundary is the static (divergence) limit, while the remaining boundaries are Hopf-type curves whose crossing frequencies are delivered by $\hat{\omega}$. The combined coloring (16) displays the interior stability margin – deep blue marks the fastest-decaying configurations – and the integer plateaus $Z = 1, 2, 3, 4$ outside, all in one map. The black MDBM curve is the high-resolution stability boundary traced from the scalar objective (17), and the red star marks the most robust gain combination found by the largest inscribed ellipse.

Figure 2 dissects the single-point computation at the two marked points. At the stable point S the locus of $\mathcal{D}(\mathrm{i}\omega)$ avoids the origin and the accumulated phase settles at $\Delta\theta = n\pi/2$, giving $\tilde{Z} \approx 0$; at the unstable point U the locus encircles the origin and $\tilde{Z} \approx 2$. Panel (b) makes the central numerical argument of this paper visible: the accepted steps of the adaptive solver concentrate at the sharp peaks of θ' near the lightly damped resonances – precisely the features a fixed frequency grid tends to miss. Panel (c) shows the same march continuing over more than two further decades of frequency: the delayed *velocity* feedback of this system produces a phase ripple decaying only like ω^{-1} (Section 3.1), so the solver keeps taking moderate steps through the tail. The chart studies below use the standard window $\omega_{\max} = 10^4$ – its truncated tail, $\approx 0.2/\omega_{\max} \approx 2 \cdot 10^{-5}$, is far below the rounding threshold, and its role in the error budget is measured explicitly – while the single-point convergence study of Section Appendix C.1 removes

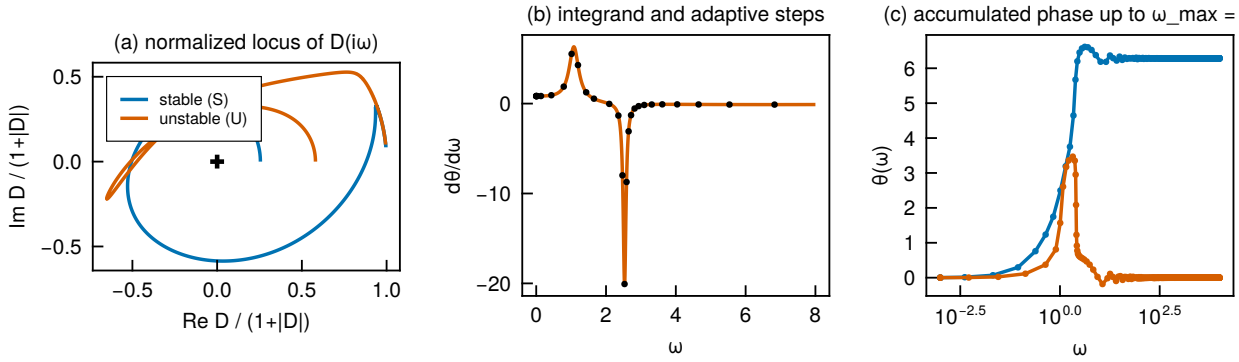


Figure 2: The method at the stable point S and unstable point U of Fig. 1. (a) Normalized locus of $\mathcal{D}(\mathrm{i}\omega)$: the stable curve avoids the origin, the unstable one encircles it. (b) Phase slope $\theta'(\omega)$ with the accepted steps of the adaptive solver (dots): steps cluster automatically at the near-singular peaks that indicate close characteristic roots. (c) Accumulated phase up to $\omega_{\max} = 10^4$ on a logarithmic frequency axis: past the resonances the solver keeps resolving the slowly decaying (ω^{-1}) delay ripple of this velocity-feedback system with moderate, growing steps.

even that floor by integrating to $\omega_{\max} = 10^6$. Either way the truncation is exposed by the integer-residual diagnostic, never hidden.

6.3. *Tolerance as a universal accuracy dial*

Computing the same chart with the phase ODE at tolerances 10^{-2}, 10^{-4} and 10^{-6} (Appendix C.2) shows that the tolerance buys *certainty*, not appearance: the three charts look alike, yet 10^{-2} miscounts 349 of the 8000 points against the 10^{-6} reference and flips the classification on 89 – a failure the residual field announces over the whole chart – while 10^{-4} leaves a single count off by one and 10^{-6} none, for a factor of three in time. While the integrator dominates the error, the largest residual of a chart tracks the tolerance,

$$\max \varepsilon \;\approx\; 100 \cdot \mathrm{tol}, \tag{22}$$

within a factor of four, the factor being inherited from the accepted-step count; below $\sim 10^{-6}$ the truncated tail at $\omega_{\max} = 10^4$ takes over, and the residual itself tells the user that the next decade must come from $\omega_{\max}$. The traced boundary, built from $\hat{\sigma}$ rather than from the rounded count, stays a small fraction of a pixel from the reference at every tolerance.

6.4. *Rightmost roots and their refinement*

Figure 3 quantifies the rightmost-root estimates with a yardstick that favors no method: the residual $|\mathcal{D}(\lambda)|$, which measures distance to an actual root, and the deviation from an independent semi-discretization solve.[1] Three facts follow. First, the raw tracked value $(\hat{\sigma}, \hat{\omega})$ already places the decay rate to $6.9 \cdot 10^{-3}$ on the smooth system and

[1] A natural but circular way to rank the polishes is to measure each against many Newton iterations taken as "truth", which trivially makes a few Newton steps look best; the metrics used here avoid that trap.

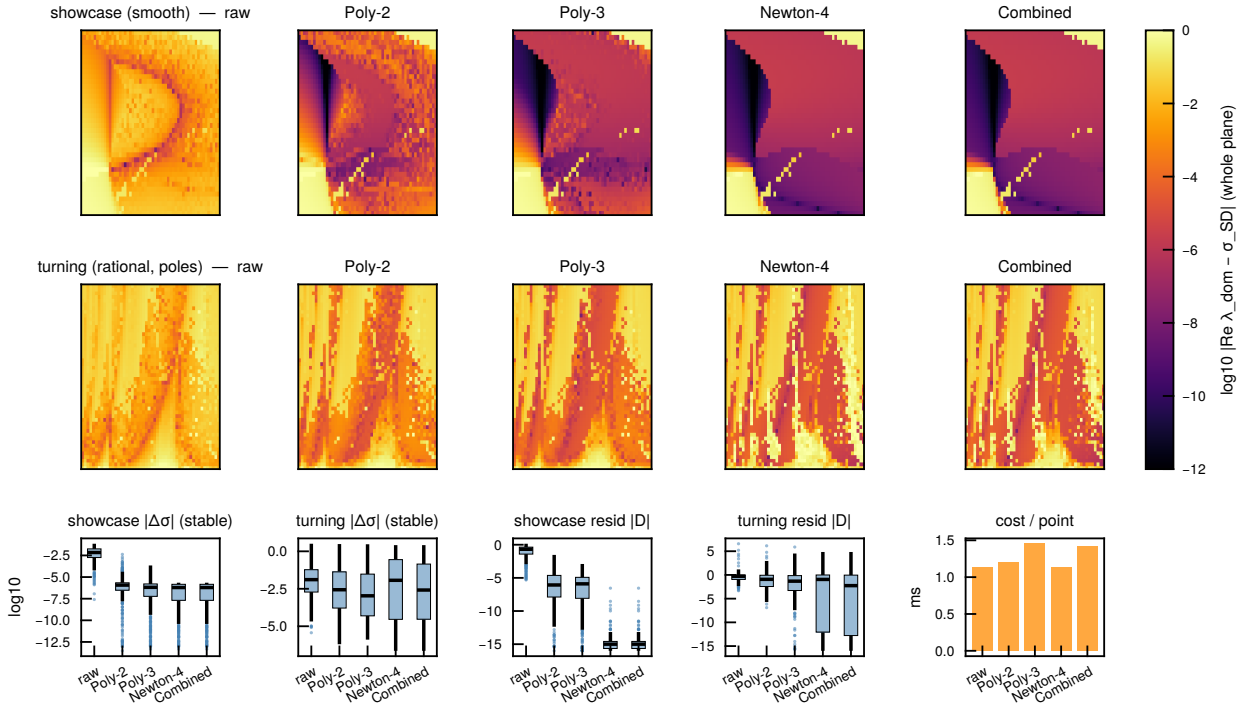


Figure 3: The optional root refinement, judged by method-independent metrics on two systems – a smooth quasi-polynomial (the showcase, top) and a rational frequency-response function with poles (the turning-lobe example of Appendix A, middle). Error *fields*: per-pixel $\log_{10}|\operatorname{Re}\lambda_{\mathrm{dom}} - \sigma_{\mathrm{SD}}|$, the deviation of the dominant root's real part from an independent semi-discretization reference, for the raw estimate and the four polishes. Error *bars* (bottom): distributions, over the stable pixels, of the residual $|\mathcal{D}(\lambda_{\mathrm{dom}})|$ (distance to an actual root) and of the same σ-deviation, for each system, together with the per-point cost. The raw estimate is the smoothest field; every polish sharpens the smooth system uniformly, while on the rational system Newton becomes patchy (it can jump to a neighboring root near a pole) and the Taylor and combined polishes stay consistent.

$1.3 \cdot 10^{-2}$ on the rational one, its sign – all a chart needs – is correct throughout, and being a single closed-form expression per pixel it is also the smoothest field, which is why it is the default. Second, no single polish wins everywhere: on the smooth system Newton drives the residual $|\mathcal{D}|$ to machine precision and saturates the accuracy of the semi-discretization reference ($6.1 \cdot 10^{-7}$), but on the rational system, where the plant response has poles near the imaginary axis, a Newton step can settle on a neighboring root and its error *worsens* to $1.2 \cdot 10^{-2}$ – the patchiness visible in its field – whereas the cubic Taylor polish stays consistent at $1.1 \cdot 10^{-3}$, and the combined mode is the robust compromise ($2.6 \cdot 10^{-3}$) that never speckles. Third, the choice is nearly free: even the combined mode adds only about 25% to the complete solve (Table 2). The practical hierarchy of Section 3.3 follows directly: raw for charts, a polish when a specific root matters, σ-contours (Section 6.5) for a precise decay-rate map. Tracking $N_r > 1$ minima returns several dominant roots at once, which the multi-root recipe of Section 7.3 exploits.

Table 2: Median error of the dominant root's decay rate Re λ against a semi-discretization reference, for each refinement, on the smooth (showcase) and non-smooth (turning) systems of Fig. 3. Times are per parameter point for the complete solve; overhead is measured against the raw (unrefined) solve.

refinement	$\lvert\Delta\sigma\rvert$ showcase	$\lvert\Delta\sigma\rvert$ turning	time / point	overhead
raw	$6.9 \cdot 10^{-3}$	$1.3 \cdot 10^{-2}$	1.1 ms	–
Poly-2	$1.2 \cdot 10^{-6}$	$2.7 \cdot 10^{-3}$	1.2 ms	+6%
Poly-3	$6.5 \cdot 10^{-7}$	$1.1 \cdot 10^{-3}$	1.5 ms	+28%
Newton-4	$6.1 \cdot 10^{-7}$	$1.2 \cdot 10^{-2}$	1.1 ms	-1%
Combined	$6.1 \cdot 10^{-7}$	$2.6 \cdot 10^{-3}$	1.4 ms	+25%

6.5. Decay-rate design: σ-contours

Finally, Fig. 4 demonstrates the γ-stability capability: the traced σ-contours (Section 4.4) enclose progressively smaller regions with larger asymptotic decay rates, and they are consistent with the interpolated $\hat{\sigma}$ background computed from the coarse sweep alone. For control design this map answers directly which gain combinations guarantee a prescribed settling rate – at the cost of one extra boundary tracing per contour.

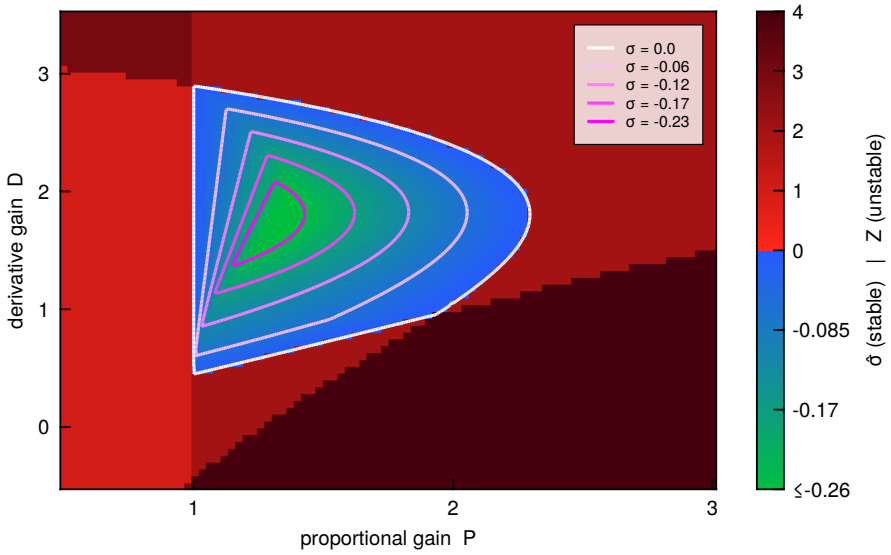


Figure 4: Decay-rate map of the showcase system: colored background – estimated decay rate $\hat{\sigma}$ from the coarse sweep of Fig. 1 (discrete march; stable domain only); curves – γ-stability boundaries traced by MDBM for a family of shifted lines $\mathrm{Re}\,\lambda = \sigma$. The two independent computations delineate the same landscape.

7. Performance analysis

7.1. The discrete march against the integration back-ends

Table 3 measures what Section 3.6 predicts. On the same machine and the same ten thousand points, the discrete march computes every chart without a single wrong count, 38, 134 and 298 times faster than the default ODE back-end, and at 1.0, 1.7 and 3.5 μs of wall time per point on 12 threads – a complete 100×100 chart of any of the three systems in 10–35 ms (the kernel of Section 8 needs about half of that per point on the same processor). The gain grows with the difficulty of the system, because the discrete march pays only for what it must resolve: the median point costs 31, 52 and 129 evaluations of $\mathcal{D}$, whereas the integrators pay for every period of the delay ripple up to $\omega_{\max}$. It is even *faster* than the fixed-step trapezoid with 500 steps, which gets 80–99 % of the counts wrong. The quadrature, which skips the root by-product, is no longer competitive anywhere. Near a boundary the discrete march is also the more robust back-end (Table 4): it reproduces the exact count down to $|P - P_b| = 10^{-12}$ on both sides of the Hopf boundary of the showcase system, spending at most 165 evaluations there, while the ODE march at the tight tolerance 10^{-8} misses the narrow peak from $P - P_b = +10^{-8}$ and -10^{-10} onwards – the failure mode discussed above, here resolved by the step rejection on an unpredicted phase jump and by the root-side decision below the resolution of the number format.

The discrete march is therefore the recommended back-end for charts of entire characteristic functions, and for neutral systems with an explicit, moderate $\omega_{\max}$ (Section 3.6); in the package it is selected explicitly (`calculate_unstable_roots_un`) while the ODE march remains the default. The ODE march is kept for what it does better: the convergence-controlled

Table 3: Complete 100×100 charts of the three benchmark systems with the integration back-ends and with the discrete phase-unwrapping march of Section 3.6, all measured in one session on one machine (Intel(R) Core(TM) i5-10400 CPU @ 2.90GHz, 12 threads), so that the speed-up factors are free of hardware differences. The integration back-ends use the chart window $\omega_{\max} = 10^4$, the discrete march $\omega_{\max} = 10^5$; the wrong-Z column counts the deviations from a tight-tolerance (10^{-8}) integration reference of the same chart. The per-point column is the chart time divided by 10^4. The turning model enters the discrete march with its denominators cleared and with the step cap $h \le 0.05$ for $\omega < 5$ (Section 3.6); its speed-up therefore includes the change of form and of window.

system	back-end	chart (100×100)	per point [μs]	wrong Z	speed-up
4th-order + delay	ODE Vern9 + root (default)	360.1 ms	36.0	0	1.0×
4th-order + delay	QuadGK, Z only	159.5 ms	15.9	0	2.3×
4th-order + delay	fixed-step (500)	75.2 ms	7.5	8075	4.8×
4th-order + delay	unwrap + root (new)	9.6 ms	1.0	0	**38**×
2-DOF DAE showcase	ODE Vern9 + root (default)	2.35 s	234.5	0	1.0×
2-DOF DAE showcase	QuadGK, Z only	3.44 s	344.5	0	0.7×
2-DOF DAE showcase	fixed-step (500)	92.3 ms	9.2	8863	25.4×
2-DOF DAE showcase	unwrap + root (new)	17.5 ms	1.7	0	**134**×
two-mode turning	ODE Vern9 + root (default)	10.37 s	1037.2	4	1.0×
two-mode turning	QuadGK, Z only	27.32 s	2731.5	0	0.4×
two-mode turning	fixed-step (500)	91.4 ms	9.1	9903	113.5×
two-mode turning	unwrap + root (new)	34.8 ms	3.5	0	**298**×

integer residual of Section 3.4 as an independent check, characteristic functions with poles whose denominators cannot be cleared (the normalization pole of order 198 of the vehicle ring in Appendix A), and densely packed lightly damped modes (the 50×50 determinant, A.10). With these exceptions the charts of the showcase and of nine gallery panels are now computed with the discrete march, 6 to 150 times faster than with the phase ODE on the same machine (`data/gallery_timings.csv`). The detailed measurements of the integration back-ends – convergence, chart-level cost and the comparison of eight solvers – are collected in Appendix C; they were taken on an earlier machine, so their *ratios* remain valid, but their absolute chart times are one to two orders of magnitude above what the discrete march delivers.

How the per-point cost of the discrete march divides among CPU and GPU threads, and what that means for the chart time on any hardware, is the subject of Section 8.

Near the stability boundary.. The one situation in which the integration back-ends genuinely differ is the immediate vicinity of a stability boundary (Table 5). As the crossing root approaches the integration line, the integrand peak narrows in proportion to ΔP (measured half-width $6.6 \cdot 10^{-8}$ at $\Delta P = 10^{-6}$). By $\Delta P = 10^{-8}$ the marching solver steps clean over the peak, loses exactly $\pm\pi$, reports one root too few – and returns a success flag and a perfectly

Table 4: The boundary stress test of Table 5, repeated with the discrete march at its default tolerance (0.3 rad) and with the ODE march at the tight tolerance 10^{-8}. Bold entries are wrong counts. The last column is the number of evaluations of $\mathcal{D}$ the discrete march spends on the point.

$P - P_b$	true Z	ODE Vern9 (10^{-8})	unwrap	evaluations
$-1 \cdot 10^{-2}$	2	2	2	72
$-1 \cdot 10^{-4}$	2	2	2	102
$-1 \cdot 10^{-6}$	2	2	2	118
$-1 \cdot 10^{-8}$	2	2	2	131
$-1 \cdot 10^{-10}$	2	**3**	2	146
$-1 \cdot 10^{-12}$	2	**3**	2	165
$+1 \cdot 10^{-2}$	4	4	4	73
$+1 \cdot 10^{-4}$	4	4	4	92
$+1 \cdot 10^{-6}$	4	4	4	106
$+1 \cdot 10^{-8}$	4	**3**	4	137
$+1 \cdot 10^{-10}$	4	**3**	4	156
$+1 \cdot 10^{-12}$	4	**3**	4	157

healthy integer residual. Gauss–Kronrod survives the same test, though not by cleverness: the persistent delay ripple forces it to subdivide finely across the whole range anyway, and at the tight tolerance of this stress test (10^{-8}) that refinement still resolves the $\Delta P = 10^{-10}$ peak. At the package default the margin is thinner: there the same experiment (`data/peak_repair.csv`) shows the quadrature losing the count silently at $\Delta P = 10^{-10}$, while the repaired march of Section 6.3 – steered onto the peak by the $|\mathcal{D}|^2$ minimum rather than searching for it – still counts correctly there.

The practical consequences are mild. Landing within $\sim 10^{-8}$ of the boundary has a probability of order $10^{-8}/\Delta p$ per pixel for a grid spacing Δp, so a chart grid hits it, if ever, in a single pixel, and the damage is a count off by one, not a wild outlier. The MDBM trace is insensitive to it: its objective tends to zero at the boundary regardless of the sign, so a miscount displaces the interpolated boundary point by no more than the skipped strip's width, $\sim 10^{-8}$ in the gain – far below plotting resolution. And the stable/unstable classification is affected only where the skip flips the count to or from zero – exactly the situation in which the tracked root (still positive, $\hat{\sigma} = +4.3 \cdot 10^{-7}$ at $\Delta P = 10^{-8}$, correctly placing a root right of the axis) disagrees with $Z = 0$ and the sign test (11) fires; the parity test (12) fires as well, since $Z = 3$ lies between the true values 4 and 2. Since the root test needs the tracked root, which quadrature cannot provide, the ODE march remains the default integration back-end.

7.2. Boundary tracing

Table 6 makes the boundary-tracing argument of Section 4.2 concrete. Four refinement levels from a 20×20 mesh yield a boundary resolved as finely as a 320×320 uniform grid would resolve it, while evaluating only the

Table 5: Approaching a Hopf boundary of the showcase system at distance ΔP in the gain. The peak half-width equals the distance of the crossing root from the axis, proportional to ΔP ($6.6 \cdot 10^{-4}\ \Delta P/10^{-2}$); the tracked estimate $\hat{\sigma}$ reproduces it down to $\Delta P = 10^{-6}$, while in the last two rows it is limited by the step of the march (Section 3.2) and only its sign – a root right of the axis – remains correct. The marching solver silently loses one root once the peak falls below its step resolution, while the integer residual stays small ($\tilde{Z}$ shifts by exactly one).

ΔP	$\hat{\sigma}$ (peak half-width)	Z march	Z quadrature	ε march	ε quadrature
$1 \cdot 10^{-2}$	$6.6 \cdot 10^{-4}$	4	4	$1.6 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-4}$	$6.6 \cdot 10^{-6}$	4	4	$1.6 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-6}$	$6.6 \cdot 10^{-8}$	4	4	$1.3 \cdot 10^{-5}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-8}$	$4.3 \cdot 10^{-7}$	3	4	$9.1 \cdot 10^{-6}$	$4.1 \cdot 10^{-7}$
$1 \cdot 10^{-10}$	$3.0 \cdot 10^{-6}$	3	4	$8.3 \cdot 10^{-6}$	$5.2 \cdot 10^{-7}$

Table 6: What the MDBM boundary trace buys: from an initial mesh with the listed refinement levels, the traced boundary has the equivalent resolution of a uniform grid of the given size, at the listed number of function evaluations. The saving factor is the ratio of the two point counts.

system	initial mesh	levels	equiv. resolution	evaluations	boundary pts	time	saving
2nd-order, no delay	20×20	4	320×320	1924	405	32.6 ms	53×
4th-order + delay	20×20	4	320×320	1958	417	113.1 ms	52×
2-DOF DAE showcase	20×20	4	320×320	2969	686	677.7 ms	34×

cells that actually bracket the boundary – a saving of 34 to 53 times in function evaluations, which is precisely why the high-resolution boundary is affordable on top of a deliberately coarse colored background. The saving matters most where an evaluation of $\mathcal{D}$ is expensive – large determinants, automatic extraction – since for a cheap scalar $\mathcal{D}$ a brute-force grid of that density is itself affordable (Section 8); and the traced boundary is a curve, which a grid of any density only approximates.

7.3. Comparison with semi-discretization

Semi-discretization [9] – here in its efficient library implementation with the multiplication-free variant [25, 26] – approximates the dominant characteristic exponent through the spectral radius of a finite transition matrix, converging polynomially in the number of discretization steps per delay. Table 7 compares it with the present method on the reduced showcase system at a stable, a near-boundary and an unstable point; the dominant root of the present method is obtained by the multi-root recipe of Section 3.2 (ten tracked minima, all refined, maximal real part); the discrete march gives the same accuracy at these points – raw errors of $2 \cdot 10^{-5}$ to $4 \cdot 10^{-2}$ with ten tracked minima, 10^{-16} to 10^{-10} after a Newton polish – whereas a single tracked minimum is not the dominant root at any of the three points, by either back-end (`data/unwrap_checks`). That both converge towards the same residual-certified reference validates the two formulations against each other. The substantive difference lies elsewhere: semi-discretization requires the user to choose the number of steps per delay *by hand*, and an insufficient resolution returns a wrong exponent without

an internal indicator, whereas the present method sets its own frequency steps and exposes part of its accuracy through the integer residual and the consistency tests (11) and (12) – although it, too, has resolution parameters ($\omega_{\max}$, the tolerance) chosen by the user. Spectral collocation of the infinitesimal generator [16, 22] converges spectrally rather than polynomially and is the stronger competitor for the dominant root of a small system; a quantitative comparison with it is left for future work. The comparison is also intentionally asymmetric in what the methods deliver: semi-discretization returns the full dominant multiplier cluster and extends unchanged to time-periodic systems, where the argument-principle formulation does not apply; the present method returns exactly what an *autonomous* stability chart needs – the root count, the boundary, and the dominant root on demand – at interactive speed.

Table 7: Comparison with semi-discretization (`SemiDiscretizationMethod.jl`, orders 1 and 2, n steps per delay) on the reduced showcase system: error of the dominant characteristic exponent and CPU time per parameter point. The reference is the Newton-polished dominant root of the quasi-polynomial itself, certified independently of either sweep by its recorded defect $|\mathcal{D}(\lambda_{\mathrm{ref}})| < 10^{-12}$. The unrefined rows report the accuracy of the tracked estimate alone; the refined rows share the polishing machinery with the reference, so their error reflects the polish rather than the sweep.

point	method / setting	error of Re λ_{dom}	time / point
stable	SD order 1, $n = 20$	$2.9 \cdot 10^{-5}$	5.1 ms
	SD order 1, $n = 160$	$4.6 \cdot 10^{-7}$	457.9 ms
	SD order 2, $n = 20$	$2.0 \cdot 10^{-6}$	6.5 ms
	SD order 2, $n = 160$	$4.0 \cdot 10^{-9}$	477.7 ms
	this work, tol 10^{-5}, unrefined	$2.7 \cdot 10^{-4}$	1.3 ms
	this work, tol 10^{-5}	$6.2 \cdot 10^{-17}$	1.4 ms
	this work, tol 10^{-8}	$3.1 \cdot 10^{-17}$	3.0 ms
near boundary	SD order 1, $n = 20$	$9.3 \cdot 10^{-5}$	5.3 ms
	SD order 1, $n = 160$	$1.5 \cdot 10^{-6}$	449.8 ms
	SD order 2, $n = 20$	$4.0 \cdot 10^{-6}$	5.5 ms
	SD order 2, $n = 160$	$7.9 \cdot 10^{-9}$	515.2 ms
	this work, tol 10^{-5}, unrefined	$2.7 \cdot 10^{-4}$	1.7 ms
	this work, tol 10^{-5}	$6.9 \cdot 10^{-18}$	1.8 ms
	this work, tol 10^{-8}	$6.9 \cdot 10^{-18}$	4.1 ms
unstable	SD order 1, $n = 20$	$9.7 \cdot 10^{-5}$	9.7 ms
	SD order 1, $n = 160$	$1.5 \cdot 10^{-6}$	434.1 ms
	SD order 2, $n = 20$	$2.5 \cdot 10^{-7}$	9.3 ms
	SD order 2, $n = 160$	$5.6 \cdot 10^{-10}$	447.3 ms
	this work, tol 10^{-5}, unrefined	$3.6 \cdot 10^{-2}$	1.2 ms
	this work, tol 10^{-5}	$5.5 \cdot 10^{-11}$	1.2 ms
	this work, tol 10^{-8}	$4.1 \cdot 10^{-12}$	2.8 ms

8. Parallel execution: from one core to a graphics processor

Absolute chart times depend on the hardware. A transferable description separates the cost of one parameter point on one thread from how that cost divides among many threads; this section measures both, on CPUs and GPUs, and shows that the chart time is proportional to $1/n$ in the number of threads n over several decades.

8.1. Why the sweep parallelizes

The parameter points of a chart are independent, and the discrete march adds three properties that a parallel machine rewards: its state is a fixed handful of scalars (no allocation), every step is the same short sequence of operations (one dual-number evaluation of $\mathcal{D}$, a comparison, a step update), and its cost per point varies only moderately (median 31, 52 and 129 evaluations for the benchmarks of Table 3, mean 32, 53 and 139). The march was therefore written once as a `KernelAbstractions.jl` kernel that compiles for a multithreaded CPU and for a GPU with one thread per point; all measurements of this section use it. (It costs about half the per-point time of the package's general CPU implementation: 0.49 against 0.96 μs on the same twelve threads.)

8.2. Measured scaling

Figure 5 shows the time of a chart of 10^6 points as a function of the number of threads, and Table 8 extracts the characteristic numbers. On the CPU the result is what the independence of the points promises. One thread of the server needs 3.8 μs per point (Float64, i.e. double precision, scalar code); up to 16 threads every doubling of the threads almost halves the time (15.0 times faster at 16), and 48 threads are 38 times faster – the deficit at the top end being consistent with simultaneous multithreading, where two hardware threads share one core. The six-core desktop follows the same line up to its six cores (5.6 times) and reaches 8.7 times with its twelve hardware threads. A chart that takes 3.8 s on one core (10^6 points of the fourth-order benchmark) thus takes 0.1 s on the 48-thread server.

On the GPU a single thread is slower – 21–32 μs per point in Float32, five to eight times the Float64 CPU time – because it runs at a lower clock and issues its instructions in order, waiting for each result. Within the first warp the threads gain less (16 threads are 7.4–7.5 times faster than one), since the threads of a warp share one instruction stream. Beyond it the time falls in exact proportion to $1/n$ over three decades, from 64 to some 10^4–10^5 threads (dotted line in Fig. 5), at a parallel efficiency $t_1/(n\,t(n)) = 0.27$–0.42, and levels off once every multiprocessor is full – on the RTX PRO 6000 near $2.9 \cdot 10^5$ resident threads. (In Float64 the curves flatten between 64 and 256 threads, while one block saturates the few double-precision units of one multiprocessor.)

8.3. A performance model

The measurements condense into a model with two hardware numbers and one efficiency. If one thread needs t_1 per point and the device sustains $P_{\mathrm{eff}} = t_1/t_{\mathrm{sat}}$ concurrent marches, a chart of N points on n threads takes

$$T(N, n) \approx \frac{N\,t_1}{\min\{\eta\,n,\ P_{\mathrm{eff}}\}}, \tag{23}$$

Table 8: The quantities behind Fig. 5: the cost of one point on one thread (64×64 chart), the per-point cost of the saturated device (2048×2048 chart, work-queue schedule), and their ratio, the *effective parallelism* $P_{\text{eff}} = t_1/t_{\text{sat}}$. *For the GPUs in Float32, P_{eff} divided by the number of CUDA cores. With one thread per point instead of the work queue the GPUs are 1.3–1.9 times faster (Table 9).

hardware	format	one thread [μs/point]	saturated [ns/point]	effective parallelism	threads or CUDA cores (ratio*)
CPU desktop, i5-10400 (6 cores)	Float64	4.24	485	8.7	12 threads
CPU server (G4 host, 48 threads)	Float64	3.79	100	38.0	48 threads
GPU T4	Float32	30.0	6.29	4 763	2560 (1.86)
GPU T4	Float64	214.8	143.29	1 499	2560
GPU L4	Float32	21.2	2.37	8 929	7424 (1.20)
GPU L4	Float64	172.7	77.76	2 221	7424
GPU A100	Float32	31.8	1.80	17 637	6912 (2.55)
GPU A100	Float64	60.9	5.37	11 345	6912
GPU RTX PRO 6000	Float32	20.7	0.48	42 953	24064 (1.78)
GPU RTX PRO 6000	Float64	271.6	20.03	13 563	24064

with $\eta \approx 1$ on a CPU up to its physical core count and $\eta \approx 0.27$–0.42 on the GPUs in Float32 for $n \geq 64$. On a CPU, P_{eff} is the number of cores plus a fraction for hardware threads. On the four GPUs it is 1.2 to 2.6 times the number of CUDA cores in Float32 (Table 8). It exceeds the core count because a lone thread is limited by instruction latency, whereas a saturated multiprocessor issues an instruction in every cycle from one of its resident warps. The march is bound by instruction issue, not by memory: a point writes some twenty bytes, a few percent of the memory bandwidth even on the fastest card, and the L4 is 2.7 times faster than the T4 at almost the same bandwidth. The saturated cost times the core count, $t_{\text{sat}} \times$ cores, varies by less than a factor of 1.5 between the four cards, so the core count and a per-point cost measured once estimate the chart time on another NVIDIA card within that factor; hardware of other vendors, reachable through `KernelAbstractions.jl`, was not tested. P_{eff} also carries over between problems: the two-mode turning model, with 4.4 times the evaluations of the fourth-order benchmark, each 1.4 times as expensive, needs 100 μs per point on one thread of the RTX PRO 6000 and 3.0 ns per point saturated, an effective parallelism of 33 500 against 42 953. For an interactive loop a fixed per-frame cost – kernel launch and the transfer of the result – adds to Eq. (23).

Float64 follows the same model with a larger t_1, and Fig. 5 shows the consequence of the hardware design. The graphics-class chips of the T4, the L4 and the RTX PRO 6000 execute double-precision operations at 1/32–1/64 of the single-precision rate, so their saturated Float64 time is 23–42 times the Float32 time (only part of the instruction stream is double precision), whereas the A100, built for scientific computing, pays a factor of three. On this benchmark the Float32 chart equals the Float64 chart except at a single flagged point of two million (Table 9), so Float64 serves as

Table 9: Brute-force charts of the fourth-order benchmark with the discrete march, one GPU thread per point, on four NVIDIA GPUs (Google Colab): median wall time of the kernel call in milliseconds, including launch and synchronization (full HD, $1920 \times 1080 \approx 2.07 \cdot 10^6$ points, ten repetitions; 8K, $7680 \times 4320 \approx 3.3 \cdot 10^7$ points, Float32, $\omega_{\max} = 10^5$, five repetitions). $\omega_{\max} = 10^5$ is the production window; $\omega_{\max} = 15$ suffices for this model and is required by Float16, whose range ends at 65 504; in the Float16 column $\mathcal{D}$ is evaluated in Float16 on a rescaled frequency axis while the march runs in Float32. The throughput column, in million points per second, refers to Float32. The timings of the T4 (passively cooled, 70 W) scatter by up to a factor of two between cases under its thermal clock control – its 8K chart costs half as much per point as its full-HD chart; at 8K the median exceeds the minimum by 18–28 % on the A100 and the L4 as well.

	full HD, $\omega_{\max} = 10^5$			full HD, $\omega_{\max} = 15$		8K
GPU	Float64	Float32	[Mpts/s]	Float32	Float16	Float32
T4	182	**14.5**	143	3.82	11.1	104
L4	98.2	**2.67**	778	1.84	1.96	49.9
A100 (40 GB)	8.44	**2.79**	743	1.91	2.01	40.0
RTX PRO 6000 (Blackwell)	27.3	**0.746**	2778	0.508	0.565	10.3

a reference on the GPU rather than as a production format. Float32 is adequate as long as the phase error of the delay terms, $\epsilon_{32}\,\omega_{\max}\tau_{\max}$ ($3 \cdot 10^{-3}$ rad here), stays far below the step tolerance of the march; flagged points are re-checked in Float64.

8.4. Real-time charts at 8K resolution

The horizontal lines of Fig. 5 translate the per-point cost into frame rates. A chart refreshed thirty times per second leaves 33 ms per frame; for a full-HD chart ($2.07 \cdot 10^6$ points) this needs 16 ns per point, and for an 8K chart ($3.3 \cdot 10^7$ points) 1 ns per point. A single CPU core is more than two orders of magnitude above the first line, and even the 48-thread server is a factor of six above it: a full-HD chart takes 0.2 s, five frames per second, and thirty frames per second would require about one sixth of the pixels (800×430). All four GPUs are below the full-HD line. Only the fastest, the RTX PRO 6000 (Blackwell), is below the 8K line as well: at 0.48 ns per point on the work queue of Fig. 5 (1.0 ms for full HD, 16 ms for 8K), and with the production schedule of one thread per point at 0.746 ms for the full-HD chart and 10.3 ms for the 8K chart in Float32 (Table 9). The other cards compute the 8K chart in 40–104 ms. For comparison at equal precision, the Float64 march on the RTX PRO 6000 (Blackwell) is 189 times faster than one Float64 CPU thread and 5.0 times faster than the whole 48-thread server; in Float32 it is 207 times faster than the server.

On this card the chart can therefore be recomputed *while a slider is being moved*: the delay, a gain or a damping ratio is changed, and the complete chart – at 8K more than thirty million stability decisions, each with an estimate of the root nearest the imaginary axis – is recomputed within the time of one video frame. What then limits the frame rate is the transfer: copying the result fields to the host costs 4.12 ms per frame in full HD but 78.2 ms at 8K, so an interactive application keeps the result on the device and transfers only the colored image (about 2 ms per frame at

8K, `gpu/results/interactive_G4.csv`).

These frame rates belong to the given problem. The cost per point is proportional to the cost of one evaluation of $\mathcal{D}$ and to the number of evaluations, so a model whose $\mathcal{D}$ is a large determinant or whose phase march needs more steps moves all curves of Fig. 5 upwards by the corresponding factor: on the same card the DAE showcase (1.2 ns per point) runs at about 24 frames per second at 8K, and the two-mode turning model (3.0 ns per point) at about ten frames per second at 8K and above 150 at full HD. What does not change is the slope: whatever the characteristic function, the chart time is proportional to $1/n$, from a single core to tens of thousands of GPU threads.

Number formats.. Since the march needs only the *phase* of $\mathcal{D}$ to a fraction of a radian, Float32 is the production format on every card: on this benchmark it reproduces the Float64 chart except at a single flagged point. Half precision is accurate wherever no root is close to the line – all of its rare wrong counts are flagged and restored by an in-place Float32 re-check – but it is not faster, and the 8-bit and 4-bit formats are useless for stability charts (Appendix C.5). One thread per point is also 1.3–1.9 times faster than a dynamic work queue on every card, since the cost per point varies only moderately.

8.5. From the browser to the cloud

The same march runs wherever a parallel processor is at hand, and two deployments mark the range. At the low end, the march is also implemented as a WebGPU compute shader that runs in any current web browser on the viewer's own graphics hardware, with the characteristic function typed as text and compiled to the shader on the fly. Even on an integrated laptop GPU (Intel UHD 630) the fourth-order benchmark is charted at 960×540 points in about 26 ms, i.e. interactively at screen resolution, and on the 24 validation grids of its example systems (160×90 points each) not a single count differs from the Float32 run of the Julia implementation.[2]

At the high end, rented cloud GPUs extend the brute-force sweep beyond two parameters. The cost per point does not depend on how many parameters are varied – they enter only the evaluation of $\mathcal{D}$ – so a scan of d parameters with m values each takes $T \approx m^d\, t_{\mathrm{sat}}$ by Eq. (23). With the saturated cost of the RTX PRO 6000 (Blackwell) (0.31 ns per point for the fourth-order benchmark, from the 10.3 ms 8K chart of Table 9; 3.0 ns for the two-mode turning model), a scan with $m = 100$ values per parameter takes 0.03–0.3 s in four dimensions, 3–30 s in five and 5–50 min in six, and a six-dimensional scan with $m = 200$ about 5.5 h for the benchmark. A brute-force search of a four- to six-dimensional design space – for the most robust point, or for every parameter combination meeting a decay-rate requirement via the shifted line – thus becomes a matter of minutes to hours on a single cloud GPU, before any boundary tracing or adaptive refinement is used to reduce it further.

[2]The browser application is free to use without installation at `https://bachrathyd.github.io/InterpolatedNyquist.jl/webgpu/`.

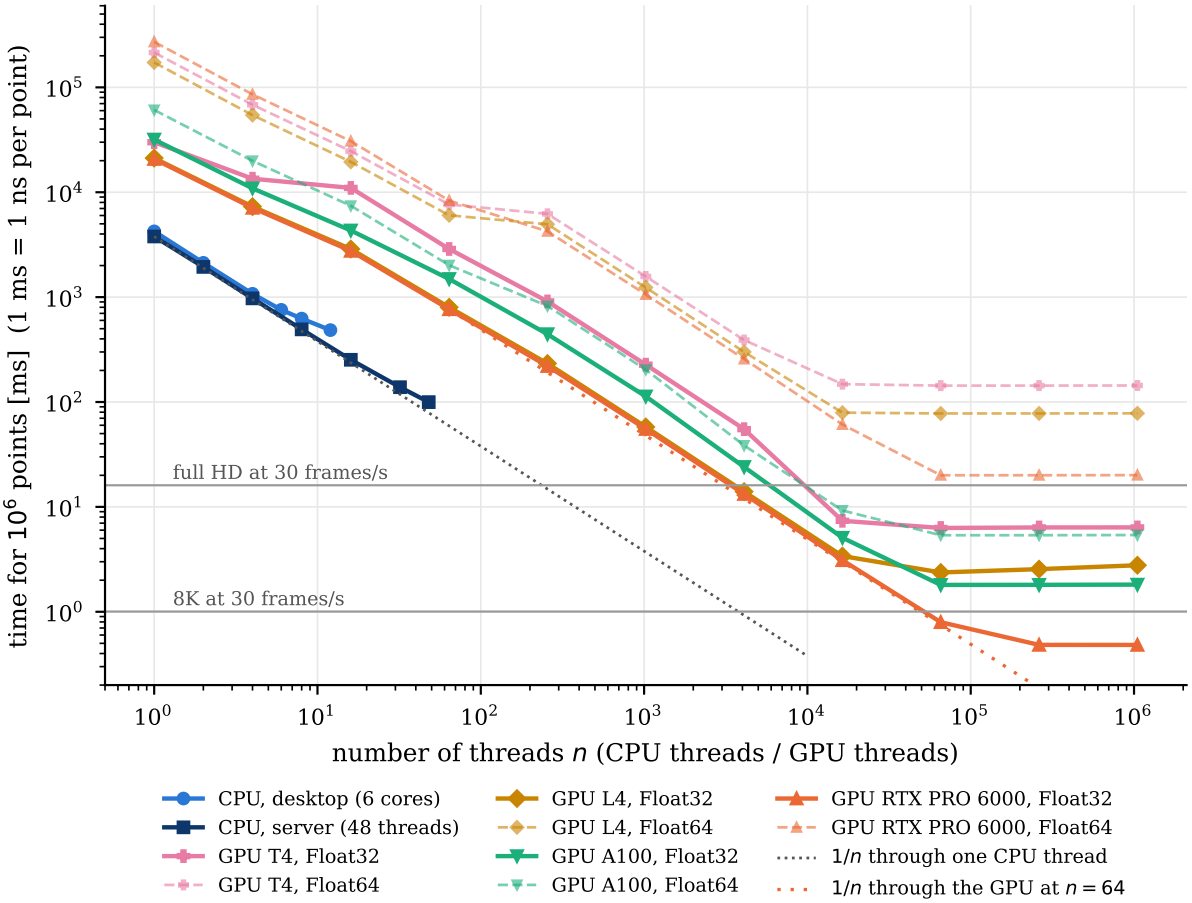


Figure 5: Parallel scaling of the discrete march on the fourth-order benchmark (Appendix A): time for 10^6 parameter points – so that 1 ms corresponds to 1 ns per point – versus the number of threads n, on two CPUs (Float64; a six-core Intel i5-10400 desktop and the 48-thread host of the Colab G4 instance) and on four NVIDIA GPUs (Google Colab), where exactly n GPU threads march the chart (solid: Float32, dashed: Float64). On the GPUs the n threads draw points from a shared work queue, in blocks of $\min(256, n)$ threads; the chart grows from 64×64 to 2048×2048 points with n, so that each thread marches at least 64 points up to $n = 65\,536$; each value is the best of three runs. The dotted lines are $1/n$ through the single-thread CPU time and through the RTX PRO 6000 at $n = 64$; the horizontal lines mark the per-point cost at which a full-HD or an 8K chart is computed thirty times per second. The T4 curve is affected by the clock control of this passively cooled card.

9. Conclusions and outlook

The Nyquist/argument-principle criterion for linear autonomous time-delay systems was turned into a fast and interpolable stability-chart tool. Marched as a scalar ODE for the phase with an exact, automatically differentiated integrand, the winding integral inherits the step-size control of modern ODE software, and the integer residual of the count gives a dimensionless error measure that follows the requested tolerance ($\max \varepsilon \approx 100\,\mathrm{tol}$). Since the count needs only the end value of the phase, the integration can also be dropped: the discrete phase-unwrapping march sums exact phase increments between adaptively placed samples, rejects steps that hide a root transition, and decides transitions below the resolution of the number format by the side of the tracked root. With 30–130 evaluations for a median point it computes the benchmark charts 38–298 times faster than the phase ODE without a single wrong count, and it resolves the Hopf boundary of the showcase down to a parameter distance of 10^{-12}.

The march is a loop of identical scalar steps, so its cost of $3.8\,\mu$s per point on one CPU thread divides almost exactly among CPU threads and, on GPUs, in proportion to $1/n$ up to the effective parallelism of the device. The fastest GPU measured computes a full-HD chart in under a millisecond and an 8K chart in about ten milliseconds – fast enough to recompute the chart while a slider is moved – and Float32 suffices. The same kernel runs in any web browser on the viewer's own graphics hardware and on rented cloud GPUs, where brute-force scans of four- to six-dimensional parameter spaces take minutes to hours.[3]

The root estimate obtained during the same sweep makes the integer root-counting map interpolable: one coloring shows the number of unstable roots outside and the stability margin inside the stable domain; the estimated root distance drives the multi-dimensional bisection method along the true stability boundary (not along all D-curves); the largest inscribed circle yields the most robust parameter point; and shifted lines yield decay-rate contours. The characteristic function is extracted automatically from a simulation-ready right-hand side, including nonlinear models and singular mass matrices, and the identical workflow was demonstrated on retarded, neutral, distributed-delay, differential–algebraic, transcendental, high-dimensional and fractional-order systems. One modeling lesson costs nothing and outweighs any tuning of the numerics: where a model admits an open-loop/feedback split, the return difference $1 + L(\lambda)$ has the same zeros, adds only open-loop poles, and has the exact order $n = 0$ – for the phase ODE; the discrete march needs the denominators cleared again.

The limitations delineate the outlook. The method is restricted to autonomous linear systems; for time-periodic coefficients time-domain methods such as semi-discretization [9, 26] remain the practical choice. Away from the boundary the root estimate is of first order unless refined, so the method complements full spectral solvers when the deep spectrum is needed. A skipped root transition leaves the count an integer; the cross-check with the tracked root, the parity test and the step rejection of the discrete march detect or avoid it, but a pair of transitions in the

[3] Both are ready to use. The browser application is free and needs no installation: `https://bachrathyd.github.io/InterpolatedNyquist.jl/webgpu/`. The Google Colab notebooks in `gpu/colab/` of the repository run the GPU kernel – optionally behind the same browser interface – on a cloud GPU of the user's own Colab account, billed in its compute units.

same direction within one step remains possible near double-Hopf points, and a guaranteed count would require a different device, such as interval arithmetic [60]. For neutral systems beyond the essential instability threshold the count diverges with $\omega_{\max}$, although the stable/unstable classification remains correct; the fractional-order results are preliminary, in particular for shifted lines; and a quantitative comparison with spectral collocation of the infinitesimal generator is still to be made. For large models the cost is dominated by each evaluation of $\mathcal{D}$; the matrix determinant lemma (3), cached factorizations, large determinants on the GPU, neutral right-hand sides in the automatic extraction, and coupling the real-time charts to optimization loops are natural continuations.

Software and reproducibility

The method is implemented in the open-source Julia package `InterpolatedNyquist.jl` (MIT license, `https://github.com/bachrathyd/InterpolatedNyquist.jl`), built on `OrdinaryDiffEq.jl` [71], `ForwardDiff.jl` [72], `QuadGK.jl` [77], `MDBM.jl` [51] and `DelaunayTriangulation.jl` [78]. The GPU implementation of the discrete march (`NyquistGPU`, with Google Colab notebooks) is in the `gpu/` directory and the WebGPU browser application in the `webgpu/` directory of the same repository. All figures and tables of this paper can be regenerated by the scripts in the `paper/` directory; the benchmark hardware and software versions are embedded automatically in the manuscript.

Funding

The research leading to these results was supported by the Hungarian Scientific Research Fund (NKFIH-AG-152125).

Declaration of competing interest

The author declares that there is no known competing financial interest or personal relationship that could have appeared to influence the work reported in this paper.

Appendix A. Case-study gallery

This appendix demonstrates the identical workflow of Section 4.5 – coarse colored sweep, MDBM boundary, combined coloring (16) – on eleven further systems covering all supported model classes, plus the reproduction of a published design chart. The complete scripts, grid resolutions and timings are part of the package repository.

These charts are the speed argument.. Sections 6 and 7 make the accuracy case, with tolerances down to 10^{-8}. The gallery answers the complementary question – *what does a chart cost when one simply wants to see the picture?* – and so every panel is computed at settings chosen for speed: a 75×75 background grid (40×40 for A.10). Nine of the panels are computed with the discrete march of Section 3.6 at its default tolerance, with the step cap of that

section where a delay chain or a lightly damped structure requires it; each of these grids was checked point by point against the phase-ODE grid of the same points, and every difference was decided by a tight (10^{-10}) re-count: the discrete march was right at all of them except at points with a root exactly on the line, where no count exists (`data/unwrap_check_s08.csv`). Three panels stay on the phase ODE at a tolerance of 10^{-4}: A.2, whose coloring of the stable domain the root estimates of the march leave incomplete, the vehicle ring, whose normalization pole of order 198 cannot be cleared, and A.10 (see there); there the count of a few boundary-adjacent pixels may be off by one (Section 6.3), invisible at chart resolution, while the traced boundary – built from $\hat{\sigma}$, not from the count – is unaffected. The MDBM boundary itself is traced with the phase ODE on all panels. The MDBM traces start from a 7×7 mesh and refine until the boundary carries roughly four times the resolution of the background grid. $\omega_{\max} = 10^4$, the standard window of this paper, is used wherever it helps; five panels use less, for reasons of substance (the ripple of a neutral system, A.4–A.6, does not decay at all, so a wider window buys nothing, and for A.8–A.9 the Kelvin–Voigt damping extinguishes the integrand long before $\omega = 400$). Each panel reports the measured integer residual of its own grid (`data/gallery_timings.csv`): on the retarded panels of the discrete march the median residual is 10^{-10}–10^{-5} (only the truncated tail), on the phase-ODE panels $\sim 10^{-4}$, while on the neutral panels it is 0.07–0.15 – the irreducible tail oscillation of Section 3.4, not a symptom of truncation.

Reading the charts.. Every panel uses the combined field C of (16): outside the stable domain the color encodes the integer number of unstable roots as flat plateaus; inside, the same color axis continues below zero and encodes the estimated decay rate $\hat{\sigma} < 0$, varying smoothly. One map therefore answers three questions at once – where the system is stable, how robustly, and how many roots must be stabilized elsewhere. Following Section 3.2, the shading uses the *dominant* root (fifteen tracked minima, five for the matrix examples, unrefined, maximal real part) rather than the single closest minimum, and the same convention keeps the MDBM objective continuous across branch crossings. The black curve is the boundary traced by MDBM from (17), not a contour of the colored grid, and each panel is annotated with the back-end of its grid and the wall-clock cost of both stages (Intel i5-10400, twelve threads).

A.1 Fourth-order delayed oscillator.. $\mathcal{D}(\lambda) = c_1\lambda^4 + \lambda^2 + 2\zeta\lambda + 1 + (P + D\lambda)\mathrm{e}^{-\lambda\tau}$ with $c_1 = 0.03$, $\zeta = 0.02$, $\tau = 0.5$, swept over the control gains (P, D). This is the regression benchmark of the package (test suite and performance history) and the system of the solver comparison in Fig. C.12.

A.2 Delayed oscillator.. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda + b\,\mathrm{e}^{-\lambda/2}$ over (a, b): the textbook retarded case with both divergence and Hopf boundaries.

A.3 Distributed delay.. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda + b\,(1 - \mathrm{e}^{-\lambda\tau})/\lambda$, $\tau = 1$, i.e., a uniform delay kernel $\int_0^\tau x(t - \vartheta)\mathrm{d}\vartheta$. The kernel integral appears in closed form in $\mathcal{D}$; the removable singularity at $\lambda = 0$ is handled by the series fallback in the model code and by the origin offset of Section 3.1.

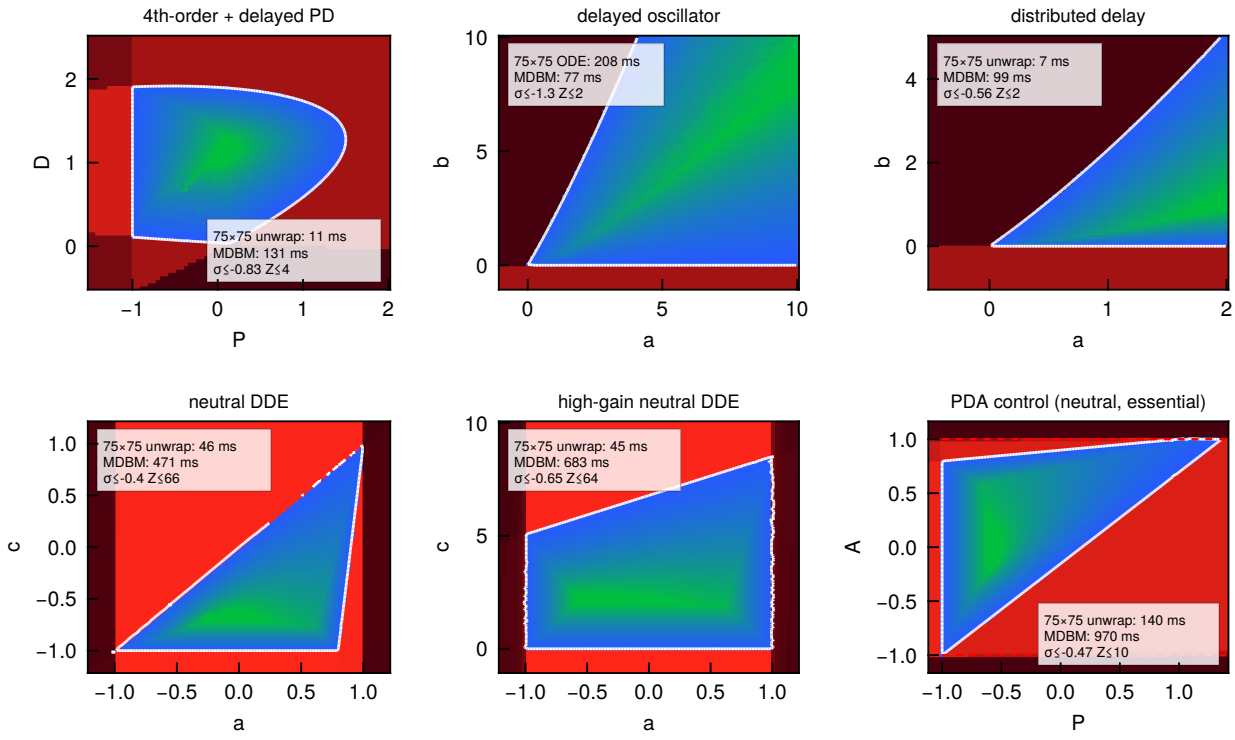


Figure A.6: Case studies, part I: (top row) fourth-order delayed oscillator, delayed oscillator, distributed delay; (bottom row) neutral DDE, high-gain neutral DDE, and PDA control with essential instability (the analytic essential boundaries $|A| = 1$ are dashed red).

A.4–A.5 Neutral systems.. $\mathcal{D}(\lambda) = \lambda^2 + a\lambda^2 e^{-\lambda} + 1 + c\,e^{-\lambda}$ and its high-gain damped variant $\mathcal{D}(\lambda) = \lambda^2 + a\lambda^2 e^{-\lambda} + 5\lambda + c\,e^{-\lambda}$, swept over (a, c) with $|a| < 1$. The delayed acceleration term makes the equations neutral: the phase integrand oscillates persistently up to arbitrarily high frequencies, so the truncation is kept moderate ($\omega_{\max} = 200$) – the resulting tail error is bounded by $\arcsin|a|/\pi < 1/2$ and never corrupts the rounded count. The order $n = 2$ is passed explicitly; the leading-order fit (7) delivers the same value, where the naive point estimator diverges. The discrete march runs with the step cap $h \le \pi/2$ (Section 3.6).

A.6 PDA control and essential instability.. Acceleration feedback is popular in active vibration control, but the delayed acceleration term $A\ddot{x}(t-\tau)$ turns the closed loop into a neutral system with characteristic function

$$\mathcal{D}(\lambda) = \lambda^2 + 2\zeta\lambda + 1 + (P + D\lambda + A\lambda^2)\,e^{-\lambda\tau},$$

whose neutral coefficient is $1 + Ae^{-\lambda\tau}$. For $|A| > 1$ the essential spectrum moves to $\mathrm{Re}\,\lambda = \ln|A|/\tau > 0$: infinitely many roots become unstable and $Z \to \infty$. With a finite $\omega_{\max}$ the computed count saturates at a large value ($\sim \omega_{\max}\tau/(2\pi)$), so the method cannot – and need not – report the exact count there; crucially, the *classification* into stable and unstable domains remains correct, and the chart in Fig. A.6 displays both the regular Hopf-type lobes and the analytic essential boundaries $|A| = 1$, which the MDBM trace locates without special treatment. The count is capped in the plot for

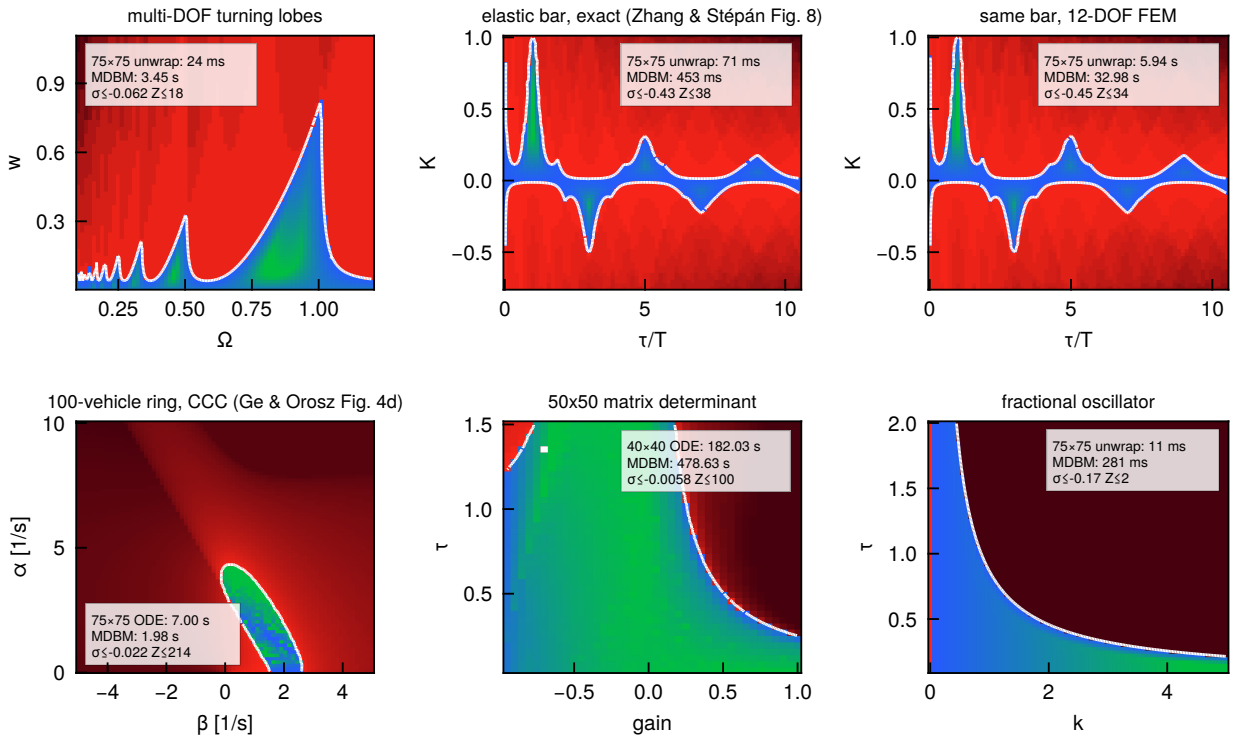


Figure A.7: Case studies, part II: (top row) multi-mode turning lobes, transcendental rod, 12-DOF finite element bar; (bottom row) ring of 100 vehicles with connected cruise control, 50×50 matrix determinant, fractional-order oscillator.

readability. This case study substantiates the claim of Section 9 that essential loss of stability is detected robustly even though the count itself diverges.

A.7 Multi-mode turning lobes.. The regenerative turning model with a two-mode flexible structure, $\mathcal{D}(\lambda) = 1 + w\,(1 - \mathrm{e}^{-2\pi\lambda/\Omega}) \sum_{k=1}^{2} g_k/(\lambda^2 + 2\zeta_k\omega_k\lambda + \omega_k^2)$, over spindle speed Ω and chip width w [1, 3, 52]. Two features deserve emphasis. First, $\mathcal{D}$ has poles – the structural poles, in the open left half-plane, so the argument-principle count is unaffected, and $\mathcal{D} \to 1$ at high frequency gives $n = 0$ for the phase ODE. The discrete march needs an entire function (Section 3.6), so for the chart the denominators are cleared, $M_1 M_2 + w(1 - \mathrm{e}^{-2\pi\lambda/\Omega})(g_1 M_2 + g_2 M_1)$ with $M_k = \lambda^2 + 2\zeta_k\omega_k\lambda + \omega_k^2$ and $n = 4$, and the step is capped at $h \le 0.05$ for $\omega < 5$ because of the regenerative root chain. Second, the chart makes the introductory argument visible: towards low spindle speeds the lobes accumulate, and a D-curve-based method would trace an unbounded family of curves of which almost none bounds the stable region, whereas the boundary objective (17) follows only the true stability limit [54].

A.8 Transcendental rod, and why the damping model decides everything.. Longitudinal vibration of an elastic bar with delayed boundary feedback gives – *without any spatial discretization* – the exact transcendental return difference

$$\mathcal{D}(\lambda) = 1 - K\,\mathrm{e}^{-r\lambda}\,\operatorname{sech}\gamma(\lambda), \tag{1}$$

in units of the wave travel time T along the bar, with the delay ratio $r = \tau/T$ and the feedback gain K, cf. the exact analysis of Zhang and Stépán [75] and the finite-DOF hierarchy of Kidd and Stépán [79]. Whether a finite count even exists, and how robust it is, is decided by the damping model. With external (viscous) damping, $\gamma = \sqrt{\lambda^2 + c\lambda} \approx \lambda + c/2$, every mode is damped equally: $\operatorname{sech}\gamma \approx 2\mathrm{e}^{-c/2}\mathrm{e}^{-\lambda}$, so $\mathcal{D}$ behaves like a neutral quasi-polynomial and the closed-loop roots accumulate along the vertical line $\operatorname{Re}\lambda = \ln(2|K|\mathrm{e}^{-c/2})/(r+1)$. The count is finite only if $2|K|\mathrm{e}^{-c/2} < 1$, and even then the chain approaches the axis as the gain grows, with a non-decaying phase ripple – the essential-spectrum situation of a neutral system (A.6). Real materials, however, dissipate through *rate-dependent* (Kelvin–Voigt) internal damping, $\gamma = \lambda/\sqrt{1+\eta\lambda}$: the high modes are damped ever harder, only finitely many roots sit near the axis, and $|\operatorname{sech}\gamma|$ decays like $\mathrm{e}^{-\sqrt{\omega/2\eta}}$. The chart uses $\eta = 0.01$, the value of [75] – a light damping that leaves the low modes and the chart shape essentially unaffected, yet is what makes the count exist at all. For the discrete march the chart counts the entire numerator $\cosh\gamma - K\mathrm{e}^{-r\lambda}$; the phase of the parameter-independent, stable denominator $\cosh\gamma$ is measured once along the line and enters as an effective order $n_{\mathrm{eff}} = 2\Phi_{\mathrm{den}}(\omega_{\max})/\pi$, which reproduces the count of the return difference exactly; the step is capped at $h \le \pi/21$ for $\omega < 40$. The chart over (r, K) in Fig. A.7 shows the resulting stable bands; the example doubles as a reminder that the right damping law is a modeling decision the method faithfully reflects rather than papers over.

A.9 Finite element bar (12 DOF), and the return difference.. The same bar discretized by 12 linear finite elements, with stiffness- proportional (Kelvin–Voigt) damping $\mathbf{C} = \eta\mathbf{K}$. Writing the delayed boundary feedback as the rank-one matrix $\mathbf{F}(\lambda) = c(\lambda)\,\boldsymbol{e}_1\boldsymbol{e}_N^{\top}$, where the scalar $c(\lambda) = -Kh^{-1}(1+\eta\lambda)\,\mathrm{e}^{-\lambda\tau}$ carries the damped modulus and the clamped-end strain sign (h = element length, $N = 12$), the naive formulation is the raw determinant $\det(\lambda^2\mathbf{M} + \lambda\mathbf{C} + \mathbf{K} + \mathbf{F}(\lambda))$, and it is a poor one: it grows like λ^{24}, reaches $\sim 10^{39}$ at the top of the integration range, and forces the counting formula onto the estimate of a large leading order – which leaves an integer residual of 0.12. Factoring out the open-loop dynamics,

$$\mathcal{D}(\lambda) = \det(\mathbf{I} + \mathbf{Q}_0(\lambda)^{-1}\mathbf{F}(\lambda)), \qquad \mathbf{Q}_0 = \lambda^2\mathbf{M} + \lambda\mathbf{C} + \mathbf{K}, \tag{2}$$

leaves the zeros untouched, adds poles only at the open-loop roots – all in the left half-plane, so the right-half-plane count is unaffected – and yields $\mathcal{D} \to 1$ at infinity, hence $n = 0$ exactly and a residual of $2.0 \cdot 10^{-6}$. Because the feedback is a single sensor–actuator pair, $\mathbf{F}$ is rank one, and the matrix determinant lemma collapses (2) to the classical scalar return difference $1 + L(\lambda)$ of (1),

$$\mathcal{D}(\lambda) = 1 + c(\lambda)\,\boldsymbol{e}_N^{\top}\mathbf{Q}_0(\lambda)^{-1}\boldsymbol{e}_1, \tag{3}$$

which replaces a full $N \times N$ solve *and* an $N \times N$ determinant by one solve with a single right-hand side: it agrees with the determinant form to $1.8 \cdot 10^{-16}$ relative and is 1.8 times cheaper per evaluation. For multi-input feedback of rank r the same lemma leaves an $r \times r$ determinant. One caveat is inherited from classical loop analysis: an exact pole–zero cancellation between the open loop and the feedback would remove that root from (2) – as it does from every input–output method – so the split should only be applied when the open-loop model is stabilizable and detectable

through the chosen loop. For the chart itself the discrete march counts the raw determinant $\det(\mathbf{Q}_0 + \mathbf{F})$, computed with one LU factorization that also serves the lemma, with the measured phase of the parameter-independent, stable $\det \mathbf{Q}_0$ as effective order – the device of A.8 – which leaves a residual of 10^{-9} instead of 0.12. The finite element chart reproduces the continuum result of A.8 within the resolved frequency range: the two are consistency checks on each other. As Section 3.1 argues, where such an open-loop/feedback split exists it is worth more than any tuning of the integrator.

A.10 Large dense determinant.. A synthetic stress test: $\mathcal{D}(\lambda) = \det(\lambda^2\mathbf{M} + \lambda\mathbf{C} + \mathbf{K} + g\,\mathrm{e}^{-\lambda\tau}\mathbf{K})$ with dense random symmetric positive definite 50×50 matrices, $\mathbf{C} = 0.25\,\mathbf{K}$, swept over a delayed stiffness gain $g > -1$ and the delay τ. (At $g = -1$ exactly, the delayed stiffness cancels the static one and a characteristic root sits *on* the integration line; the package then reports the invalid-count marker instead of an arbitrary integer.) The evaluation cost is dominated by the LU factorization per frequency; with the phase ODE a 40×40 chart takes three minutes. The discrete march is not used here: fifty lightly damped modes are packed into $\omega \in [0.34, 2.97]$, two root transitions in the same direction can fall into one step, and no tolerance or step cap that was tried removed all miscounts (it was wrong at 26 of 1600 points, the phase ODE at the chart tolerance at 11) – the generic blind spot named in Section 3.6. This example also exercises the overflow guards of Sections 3.1–3.2: $|\mathcal{D}|$ reaches 10^{250} and beyond, so the phase integrand is evaluated in scale-invariant form and the leading-order probe (7) automatically retreats to a representable s_*.

A.11 Fractional-order oscillator.. $\mathcal{D}(\lambda) = \lambda^{1.8} + 0.5\,\lambda^{0.8} + k\,\mathrm{e}^{-\lambda\tau}$ over (k, τ), with the principal branch of the fractional powers. The half-line count applies with the non-integer order $n = 1.8$, here passed explicitly (the estimate of (7) gives the same counts); the origin offset of Section 3.1 regularizes the branch point at $\lambda = 0$. For $\sigma = 0$ the integration line stays right of the branch cut and the count is unambiguous; shifted lines with $\sigma < 0$ cross the cut of the principal branch and are *not* admissible – see the discussion at the end of this appendix (after A.12) for how the violation announces itself.

A.12 Fractional controller chart from the literature.. As an external validation of the fractional capability, Fig. A.8 reproduces the stabilizing-region chart of Gao, Zhai and Liu [80] (Example 1), computed there by D-decomposition; see also the related design charts [81, 82, 83, 84]. The closed-loop characteristic function $\mathcal{D}(s) = s^{\mu}(10s^{0.5} + 1) + 5\mathrm{e}^{-0.4s}(k_p s^{\mu} + k_i)$ is a one-line model definition for the present method, and the traced boundary and the classification of the published test points agree with the reference.

Why only $\sigma = 0$, and how the violation announces itself.. Section 2.2 (assumption iii) sets out why the imaginary axis is admissible for this class while a shifted line is not: a shifted line encloses part of the branch cut of s^{μ} and the argument principle no longer applies. The violation is detectable without theory: moving the line to the left can only ever *add* roots to its right, so the γ-stable regions must be *nested* – yet a 15×15 scan of both charts returns $Z(-0.5) < Z(0)$ at 3 of 450 points, and contours drawn at several σ levels cross, which no nested family can do. This is why the stability chart reproduces the literature while a stability-*degree* chart cannot be offered at all; extending the

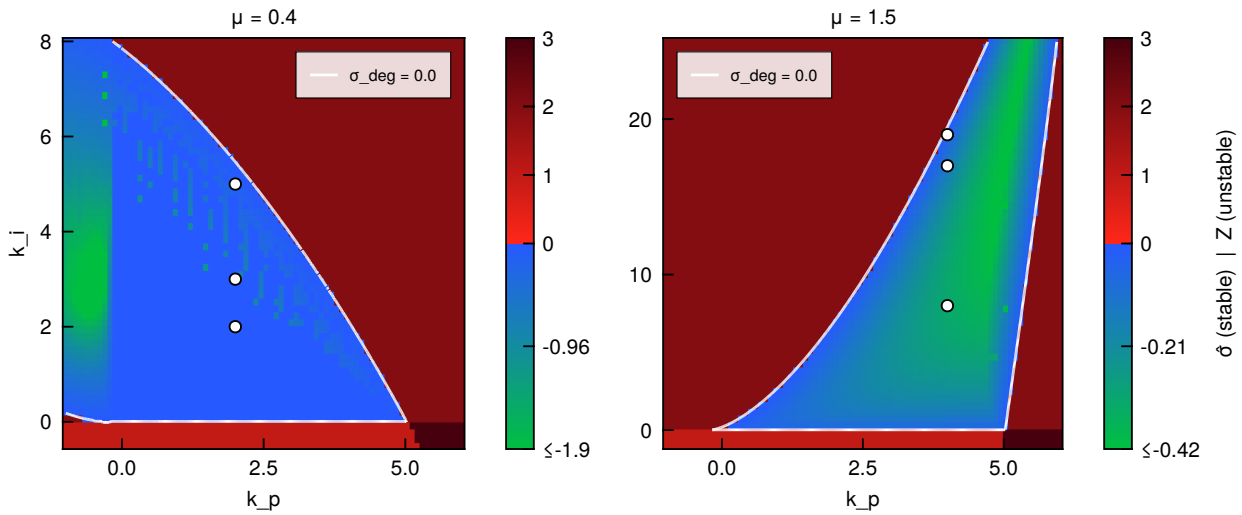


Figure A.8: Reproduction of a fractional-order controller design chart from the literature (Gao, Zhai and Liu [80], Example 1): stabilizing regions of the fractional PI controller $C(s) = k_p + k_i/s^{\mu}$ for the fractional delayed plant $G(s) = 5\mathrm{e}^{-0.4s}/(10s^{0.5} + 1)$, for $\mu = 0.4$ (left) and $\mu = 1.5$ (right). White curve: the traced stability boundary ($\sigma = 0$); white markers: the test points tabulated in [80]. Only $\sigma = 0$ is shown, and deliberately so: the shifted line of Section 2.2 is *not* admissible for this class – see the text.

criterion to fractional systems with a prescribed decay rate requires a contour that respects the cut (a keyhole, or a formulation in the $w = s^{1/m}$ plane) and is left for future work.

Appendix B. The discrete phase-unwrapping march

This appendix lists the march of Section 3.6 in the form in which it is implemented (the function `calculate_unstable_roots_unwrap` of the package, and the GPU kernel of Section 8). All quantities are evaluated on the line $\lambda = \sigma + \mathrm{i}\omega$; one call of $\mathcal{D}$ with a dual number $\omega + \delta$, $\delta^2 = 0$, returns both $\mathcal{D}$ and $\mathcal{D}_\omega = \partial_\omega \mathcal{D}$ (the $\mathcal{D}'$ of Section 3.1).

Sample.. For a sample at ω store the scale-free value $u = \mathcal{D}/s$, $s = \max\{|\operatorname{Re}\mathcal{D}|, |\operatorname{Im}\mathcal{D}|\}$ (which avoids overflow and underflow, essential in Float16), the phase slope $\theta' = \operatorname{Im}(\bar{u}\,\mathcal{D}_\omega)/(s|u|^2)$, and the sign $g = \operatorname{sgn}\operatorname{Re}(\bar{u}\,\mathcal{D}_\omega)$ of $\partial_\omega|\mathcal{D}|$.

Algorithm..

1. Set $\omega = \omega_0$ (10^{-9}), $\Phi = 0$, $h = h_0$ (10^{-2}); take the sample a at ω_0. If $g_a > 0$, $|\mathcal{D}|$ has its minimum at $\omega = 0$: record a Newton root estimate from a.
2. Trial step $h_t = \min\{h,\ h_{\mathrm{rel}}\max(\omega, 1),\ h_{\max}$ if $\omega < \omega_{\mathrm{band}}\}$, truncated at $\omega_{\max}$; take the sample b at $\omega + h_t$.
3. Observed increment $\Delta = \operatorname{Arg}(u_b\bar{u}_a)$; error $e = |\Delta - \frac{h_t}{2}(\theta'_a + \theta'_b)|$; proposal $h_{\mathrm{new}} = h_t \cdot \min\{4, \max\{0.2,\ 0.9(\mathrm{tol}/e)^{1/3}\}\}$.
4. If $e \le \mathrm{tol}$: accept. If $e > \mathrm{tol}$ but $h_{\mathrm{new}} \le 8\epsilon_{\mathrm{mach}}\max(\omega, 1)$ (sub-resolution transition): compute the Newton step $q = \mathcal{D}/\mathcal{D}_\omega$ at b; if $|\Delta| > \pi/2$, take the branch $\Delta - 2\pi$ when $\Delta > 0$ and the root lies right of the line ($\operatorname{Im} q > 0$), $\Delta + 2\pi$ when $\Delta < 0$ and the root lies left of it; accept and flag the point. Otherwise reject, set $h = h_{\mathrm{new}}$ and go to step 2.

5. On acceptance: if $g_a < 0 < g_b$, a minimum of $|\mathcal{D}|$ lies inside the step; locate it on the cubic Hermite interpolant of $\mathcal{D}$ through $(\mathcal{D}_a, \mathcal{D}_{\omega,a}, \mathcal{D}_b, \mathcal{D}_{\omega,b})$ by four regula-falsi (Illinois) iterations on $\mathrm{Re}(\bar{p}\, p') = 0$, take one Newton step from it, and insert the estimate $(\hat{\sigma}, \hat{\omega})$ into the depth-sorted buffer of the N deepest minima if it lies inside the trust region. Then $\Phi \leftarrow \Phi + \Delta$, $a \leftarrow b$, $h = h_{\mathrm{new}}$.
6. Repeat from step 2 until $\omega = \omega_{\max}$; return $\tilde{Z} = n/2 - \Phi/\pi$, $Z = \mathrm{round}(\tilde{Z})$, the root estimates, and the number of evaluations.

Parameters and flags.. The defaults are tol = 0.3 rad, $\omega_{\max} = 10^5$, $h_0 = 10^{-2}$, $h_{\mathrm{rel}} = 1$, and no band cap ($h_{\max} = \infty$). The tolerance is a statement about the phase only: it is independent of the scale of $\mathcal{D}$, and it is far from critical: every value from 0.05 to 1.2 rad reproduces the three charts of Table 3 without a single wrong count, while the cost changes by at most a factor of 2.5 between 0.1 and 0.5 – a skipped transition costs an error of almost π, well above any sensible tolerance. The GPU kernel returns three flag bits per point: *failed* (the step fell below the resolution without a decidable branch, or the step budget was exhausted), *decided* (step 4 fired) and *residual* ($|\tilde{Z} - Z| > 1/4$, typically a root on the line). Flagged points are the candidates for a re-check in a wider number format.

Appendix C. Integration back-ends, tolerance and number formats

This appendix collects the detailed measurements behind the statements of Sections 3–8 on the integration back-ends (the phase ODE, adaptive quadrature and a fixed-step trapezoid), on the solver tolerance, and on narrow number formats. The integration back-ends were measured on an earlier machine than the discrete march (Section 7.1); their ratios carry over, their absolute chart times do not.

Appendix C.1. Convergence and efficiency of the integrators

Figure C.9 and Table C.10 quantify the choice of back-end, and three points deserve emphasis. First, from the default tolerance downwards the adaptive back-ends track the requested tolerance – with the error settling roughly two orders of magnitude above it, the rule of thumb (22) of Section 6.3 – until they meet a common floor of $\approx 8 \cdot 10^{-7}$ set by the tail truncated at $\omega_{\max} = 10^6$ and the estimated leading order, not by any integrator; at loose tolerances the lower-order pairs do not yet track it (Table C.10). Second, the order of the pair matters far more than its family. Across

Table C.10: Count error $|\tilde{Z} - Z|$ (left block) and single-point CPU time (right block) of the back-ends at selected tolerances; system and point as in Fig. C.9.

tol	Vern9 (default)	Tsit5	BS3	Rosenbrock23	QuadGK	Vern9 (default)	Tsit5	BS3	Rosenbrock23	QuadGK
10^{-2}	$1.4 \cdot 10^{-2}$	$2.2 \cdot 10^{-1}$	$5.6 \cdot 10^{-2}$	$2.6 \cdot 10^{-3}$	$3.7 \cdot 10^{-3}$	19.5 ms	0.7 ms	0.2 ms	0.4 ms	86.4 ms
10^{-4}	$6.4 \cdot 10^{-3}$	$2.2 \cdot 10^{-1}$	$7.9 \cdot 10^{-2}$	$2.2 \cdot 10^{-2}$	$7.7 \cdot 10^{-7}$	79.9 ms	17.7 ms	9.3 ms	19.9 ms	240.0 ms
10^{-6}	$3.9 \cdot 10^{-5}$	$1.8 \cdot 10^{-3}$	$7.0 \cdot 10^{-4}$	$7.5 \cdot 10^{-4}$	$7.6 \cdot 10^{-7}$	139.8 ms	76.6 ms	160.0 ms	367.6 ms	239.4 ms
10^{-8}	$7.2 \cdot 10^{-7}$	$5.1 \cdot 10^{-6}$	$2.6 \cdot 10^{-7}$	$6.0 \cdot 10^{-7}$	$7.8 \cdot 10^{-7}$	238.6 ms	223.7 ms	671.3 ms	1.32 s	458.1 ms

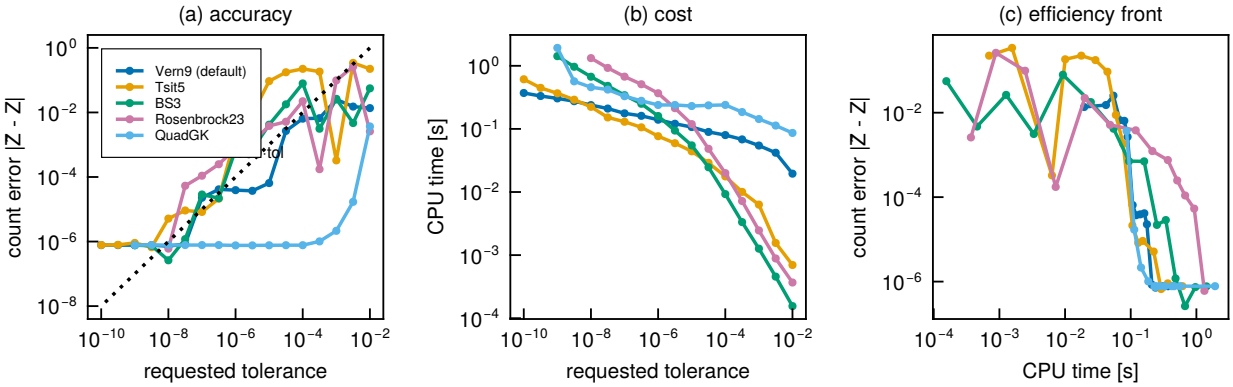


Figure C.9: Integration back-ends at an unstable point of the showcase system, where the exact count $Z = 2$ is known (verified independently by a high-resolution semi-discretization eigenvalue count), so the plotted count error $|\tilde{Z} - Z|$ is an *absolute* error measure requiring no reference. (a) Accuracy versus requested tolerance; the dotted line marks the practical rule error $\approx 100\,\mathrm{tol}$. (b) Cost versus tolerance. (c) Efficiency front. Error is on the vertical axis in both (a) and (c). A curve ends where a single parameter point exceeded a cost cap of one second, which is itself informative.

Table C.11: Tolerance study on the showcase system (grid as in Fig. C.10): sweep time, integer residual statistics, the number of grid points where the rounded count would be ambiguous ($\varepsilon \geq 1/4$), the number whose Z actually deviates from the 10^{-6}-tolerance reference, and the subset of those that flip the stable/unstable classification. The "uncertain" and "wrong" columns measure genuinely different things: a skipped peak shifts $\tilde{Z}$ by exactly one and leaves it just as close to an integer, so a point can be wrong while looking certain – which is precisely why the cross-check (11) is needed.

tolerance	CPU time	mean ε	max ε	mean/tol	max/tol	uncertain	wrong Z	flips
10^{-2}	701.0 ms	$2.7 \cdot 10^{-2}$	$5.0 \cdot 10^{-1}$	3	50	210	349	89
10^{-4}	1.30 s	$1.8 \cdot 10^{-4}$	$3.9 \cdot 10^{-2}$	2	389	0	1	0
10^{-6}	2.08 s	$1.4 \cdot 10^{-5}$	$1.4 \cdot 10^{-4}$	14	138	0	0	0

the loose half of the ladder the classical fifth-order pair [70] under-resolves the ω^{-1} ripple of this velocity-feedback system, leaving count errors of 0.2–0.34 – within a hair of the rounding threshold – and even at the package default 10^{-5} it leaves $9 \cdot 10^{-2}$ where the ninth-order default [69] leaves $6 \cdot 10^{-5}$: three orders of magnitude, for a factor of 2.4 in time. The linearly implicit Rosenbrock pair [85, 86, 87] is dominated from the default tolerance downwards, as expected once one recognizes that the problem has no stiffness for it to exploit (Section 3.1); at tight tolerances it needs seconds for a single parameter point where the ninth-order pair needs a quarter of one, which is why the sweep in Fig. C.9 stops each method once a point exceeds the cost cap. Third, at the default setting every back-end still rounds to the correct count, so the choice of integrator does not change the chart – what differs, by orders of magnitude, is the *margin* to the rounding threshold, and the default is chosen for that margin rather than for raw speed.

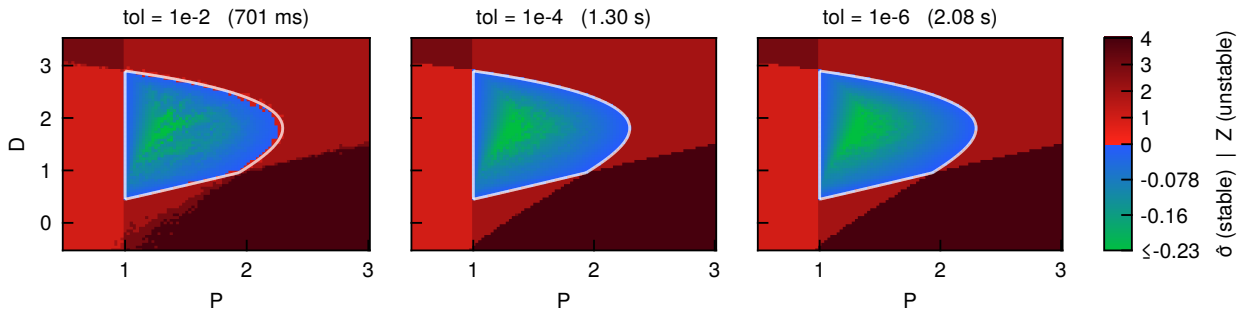


Figure C.10: The showcase chart computed with solver tolerances 10^{-2}, 10^{-4} and 10^{-6} (left to right); the wall-clock time of each full sweep is given in the panel titles. The charts are visually near-identical even at 10^{-2}, where the count is already uncertain: the tolerance buys certainty rather than appearance, cf. Fig. C.11 and Table C.11. The study uses the standard chart window $\omega_{\max} = 10^4$ and the reduced form (21) of the characteristic function.

Appendix C.2. Tolerance study on the showcase system

Figure C.10 shows the identical chart computed with tolerances 10^{-2}, 10^{-4} and 10^{-6}, together with the measured sweep times, and Fig. C.11 shows the corresponding integer-residual fields (10); the ladder deliberately starts beyond the safe range. Three observations matter for practice. First, the tolerance buys *certainty*, not appearance. The panels are visually near-identical, yet the loosest one is genuinely broken: measured against the 10^{-6} reference, 10^{-2} miscounts 349 of the 8000 points and flips the stable/unstable classification on 89 (Table C.11). That failure is anything but silent – the residual field of Fig. C.11 lights up across the whole chart, with 210 points beyond the rounding threshold – and one decade into the safe range the problem is essentially gone: at 10^{-4} a single count of the 8000 deviates, by one and without flipping the classification, and at 10^{-6} none. Second, while the integrator dominates the error budget, the largest integer residual of the chart tracks the requested tolerance with a roughly constant factor, $\max \varepsilon \approx 100\,\mathrm{tol}$, Eq. (22), to within a factor of four (Table C.11; the mean residual is one to two orders of magnitude smaller); the factor is inherited from the accepted-step count of the march. At the standard window $\omega_{\max} = 10^4$ the truncated tail of this velocity-feedback system, $\approx 2 \cdot 10^{-5}$, takes over once the tolerance drops below $\sim 10^{-6}$ – the mean residual of the tightest panel settles exactly there – and, because every error source surfaces in the same number, the diagnostic itself tells the user that the next decade of certainty must come from $\omega_{\max}$, not from the tolerance. The accuracy of Z can therefore be *prescribed* rather than discovered: a user who wants the count certain to 10^{-3} asks for 10^{-5}, which is exactly the package default. Third, the CPU-time premium of tightening by four decades is a factor of three (Table C.11), because the extra effort is spent only near the integrand peaks.

For chart work a miscount is a milder error than a displaced boundary, and the two degrade differently: the count is a rounded quantity and fails discontinuously, whereas the tracked $\hat{\sigma}$ inherits the tolerance smoothly. The traced boundary is built from $\hat{\sigma}$ alone through the objective (17), and it stays a small fraction of one grid pixel away from the reference trace at every tolerance studied (`data/boundary_shift.csv`). The practical recipe follows: a moderately loose tolerance (10^{-3}–10^{-4}) is an appropriate tool for an initial search – the residual field will announce if it was too

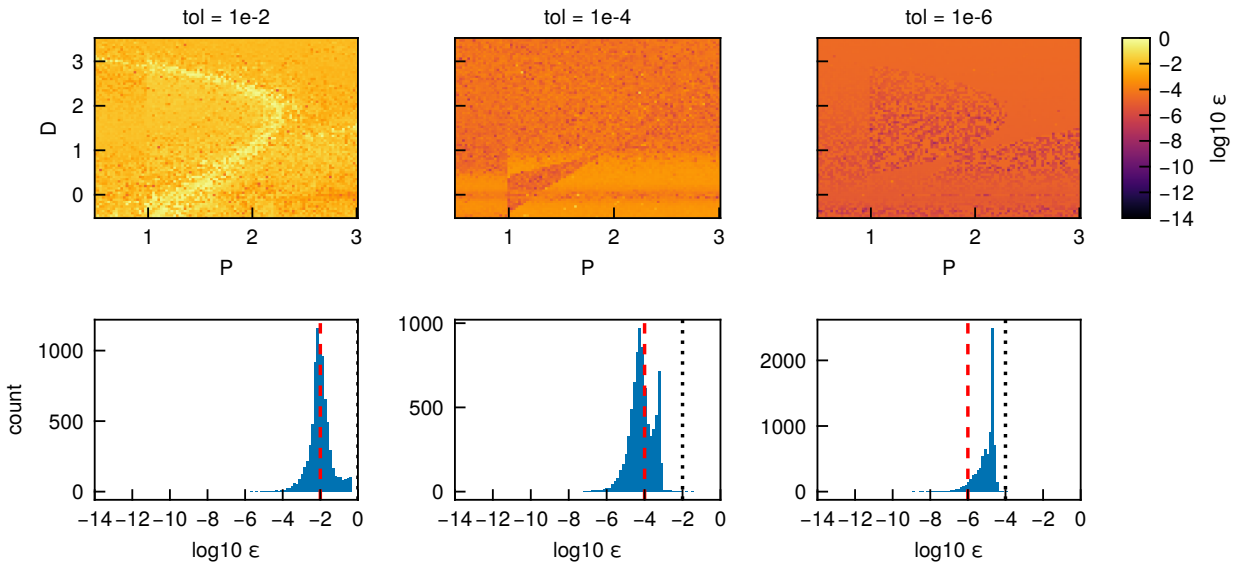


Figure C.11: Self-validating error: fields and histograms of the integer residual $\varepsilon = |\tilde{Z} - \mathrm{round}(\tilde{Z})|$, cf. (10), for the three tolerances of Fig. C.10. Dashed red line: the requested tolerance; dotted black line: the practical rule $\varepsilon \approx 100\,\mathrm{tol}$. The residual scales with the tolerance uniformly over the parameter plane, amplified by the accepted-step count of the frequency march, until the tail truncated at $\omega_{\max} = 10^4$ (of order $2 \cdot 10^{-5}$) becomes the dominant error.

loose – after which the interesting region is recomputed at the default setting to certify the counts, with the cross-check (11) guarding the one error mode near the boundary that the residual cannot reveal.

A flagged pixel need not be recomputed with a different back-end: the march can repair itself (`peak_repair=true`). The skipped peak is easy to *locate* even when it is far too narrow to *integrate*: its center is a minimum of $|\mathcal{D}(\sigma + \mathrm{i}\omega)|^2$, which dips toward that minimum over an $O(1)$ frequency range no matter how narrow the peak is. A root-finding callback stops the integrator at every such minimum, recomputes the phase increment of the crossing step by quadrature whose subdivision is forced into the located peak, and restarts the march at the minimum. Measured across the Hopf point of this chart (`data/peak_repair.csv`): the plain march loses one count from $\Delta P = 10^{-6}$ inward, the Gauss–Kronrod back-end – which must *discover* the peak – loses it silently at $\Delta P = 10^{-10}$, whereas the repaired march, which *knows* where the peak is, still counts correctly there. The repair matters only in the immediate vicinity of the boundary, and arming it for a whole sweep costs $\approx 1.9\times$ the plain march, which is why it is off by default; the economical pattern is to sweep plainly, flag with (11), and re-evaluate only the flagged pixels with the repair on.

Appendix C.3. Chart-level cost, memory and cold start

Two operating points matter in practice, and Table C.13 separates them. With the *accurate* preset – the standard chart window $\omega_{\max} = 10^4$ and the default $\mathrm{tol} = 10^{-5}$ – a complete 100×100 chart of a scalar characteristic function costs a fraction of a second, and every one of the ten thousand counts matches a tight reference. A *fast* preset ($\mathrm{tol} = 10^{-3}$) buys roughly a factor of two at the price of a few dozen boundary-adjacent miscounts – acceptable for a

Table C.12: Warm chart-level costs (median of repeated runs, 16 threads): full brute-force sweeps for three systems of increasing evaluation cost, with allocated memory. The last column counts the points at which the back-end's Z deviates from a tight-tolerance reference of the same chart: the adaptive back-ends are required to match everywhere before their time is recorded, while the fixed-step back-end – whose resolution is set by hand – reports its misses.

system	back-end	chart (100×100)	per point [μs]	alloc [MB]	wrong Z
2nd-order, no delay	ODE Vern9 + root (default)	48.2 ms	5	25	0
2nd-order, no delay	ODE Vern9, Z only	46.0 ms	5	23	0
2nd-order, no delay	QuadGK, Z only	31.8 ms	3	27	0
2nd-order, no delay	fixed-step (500)	36.4 ms	4	1	4324
4th-order + delay	ODE Vern9 + root (default)	426.2 ms	43	25	0
4th-order + delay	ODE Vern9, Z only	418.3 ms	42	23	0
4th-order + delay	QuadGK, Z only	176.2 ms	18	38	0
4th-order + delay	fixed-step (500)	96.3 ms	10	1	4556
2-DOF DAE showcase	ODE Vern9 + root (default)	1.92 s	192	25	0
2-DOF DAE showcase	ODE Vern9, Z only	1.97 s	197	23	0
2-DOF DAE showcase	QuadGK, Z only	2.72 s	272	439	0
2-DOF DAE showcase	fixed-step (500)	80.5 ms	8	1	6489

first look, and flagged by the diagnostics of Section 6.3. Beyond the presets, two user choices dominate the cost on the showcase DDAE, spanning three orders of magnitude together: $\omega_{\max}$ (the velocity feedback's ω^{-1} ripple makes $\omega_{\max} = 10^6$ cost some fifty times $\omega_{\max} = 10^4$, while buying nothing – the truncated tail is five orders below the rounding threshold) and the form of $\mathcal{D}$ (the automatic extraction costs a factor of 21 per evaluation over the hand-derived expression, Table 1). Choosing badly on both turns the 1.98 s chart of Table C.13 into one of more than half an hour; this is what the asymmetric guidance of Section 3.1 means in practice.

Table C.12 reports the cost of a *complete* 100×100 chart – ten thousand parameter points – on a AMD Ryzen 7 5800H with Radeon Graphics (16 threads, 15 GB RAM, Julia 1.12.4, package revision b37a016), with the per-point cost listed alongside so that other resolutions can be scaled directly. For scalar characteristic functions a full chart takes tens to hundreds of milliseconds, and the entire hybrid workflow (colored sweep plus MDBM boundary) finishes well inside one second – genuinely interactive. The last column is worth a glance: the fixed-step back-end is the fastest entry for the delayed systems and gets thousands of the ten thousand counts wrong, for the reason given in Section 3.5. The showcase DDAE is listed with its hand-derived characteristic function, as are all figures in this paper; what the automatic extraction of Section 5 would cost instead is measured separately in Table 1. Memory stays modest throughout because the phase ODE carries a single scalar state and the sweep stores only the five result fields; no large matrices are formed at any point. Cold start is a one-time cost dominated by package loading: in a fresh Julia process, loading takes 5–8 s and the first thousand-point sweep 0.07–0.2 s thanks to the precompilation workload

Table C.13: The two operating points of the method on the same 100×100 chart: the *accurate* preset (the default tolerance) and a *fast* preset for first exploration. The last column counts the points whose Z differs from a tight reference, i.e. the price of the fast preset, which buys roughly a factor of two in time.

system	preset	$\omega_{\max}$	tol	chart (100×100)	per point [μs]	wrong Z
2nd-order, no delay	accurate	10^4	10^{-5}	47.4 ms	5	0
	fast	10^4	10^{-3}	36.3 ms	4	0
4th-order + delay	accurate	10^4	10^{-5}	437.2 ms	44	0
	fast	10^4	10^{-3}	205.3 ms	21	14
2-DOF DAE showcase	accurate	10^4	10^{-5}	1.98 s	198	0
	fast	10^4	10^{-3}	1.18 s	118	45

Table C.14: Evaluations of $\mathcal{D}$ needed for one parameter point at tol = 10^{-5}, with the count of each entry verified against a tight-tolerance reference before being recorded. What governs the cost is how fast the delay-induced ripple of θ' decays, i.e. how far the delayed terms sit below the leading power: ω^{-1} for the velocity-feedback showcase, ω^{-3} for the fourth-order position-feedback benchmark.

system	back-end	$\omega_{\max} = 10^4$	$\omega_{\max} = 10^6$
showcase (ω^{-1} ripple)	ordered march	11 138	578 418
	quadrature	15 315	1 675 500
4th-order (ω^{-3} ripple)	ordered march	1 330	1 426
	quadrature	555	570

shipped with the package, after which every further chart runs at the warm speeds of Table C.12.

Appendix C.4. A broader solver comparison

Table C.14 isolates what actually drives the cost. The decisive quantity is not the back-end but the decay rate of the phase ripple, set by how far the delayed terms sit below the leading power of $\mathcal{D}$. Where the ripple decays fast (delayed *position* feedback, $\theta' \sim \omega^{-3}$), quadrature recognizes the negligible tail panel by panel and finishes about 2.5 times cheaper than the march; where it decays slowly (delayed *velocity* feedback, $\theta' \sim \omega^{-1}$), the ranking reverses and the march is some three times cheaper. Neither back-end dominates, and the lesson for a user is to spend attention on $\omega_{\max}$ and on the structure of the feedback, not on the choice of integrator. One implementation detail underpins the quadrature numbers: the Gauss–Kronrod back-end is seeded with logarithmically spaced interior breakpoints, one per two frequency decades. Without them the first G_7–K_{15} panel spans the entire window, its lowest node lands above the whole resonance region for $\omega_{\max} = 10^6$, and the routine returns $\tilde{Z} = n/2$ having never seen a single root – with an integer residual of *exactly* zero, since nothing about the quadrature failed. The breakpoints cost a few dozen extra evaluations and remove the failure mode entirely.

Figure C.12 extends the comparison to a wider selection of ODE solvers on the fourth-order benchmark system.

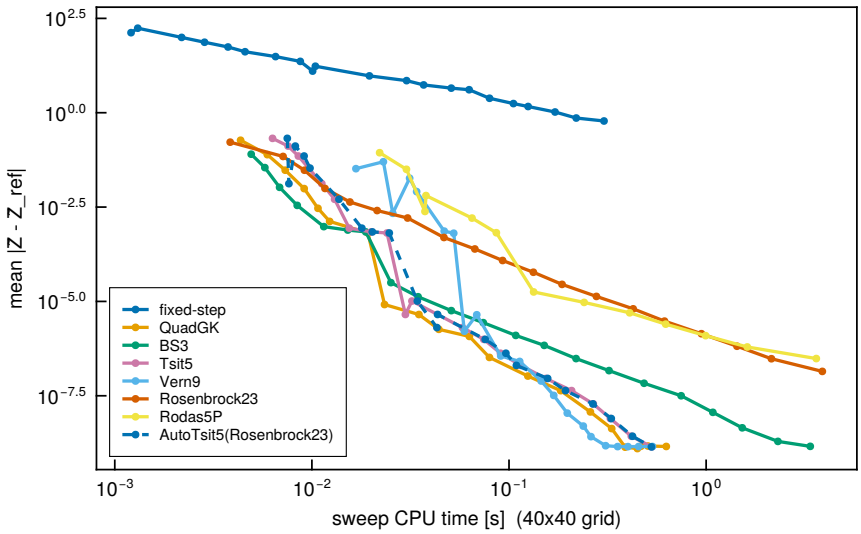


Figure C.12: Error–cost Pareto fronts of eight integration back-ends on a 40×40 sweep of the fourth-order benchmark oscillator (Appendix A): fixed-step trapezoid (step-count sweep), adaptive Gauss–Kronrod, and six ODE solvers (tolerance sweeps).

The fixed-step trapezoid is *dominated everywhere*: even at $3 \cdot 10^4$ steps – more than an order of magnitude slower than the adaptive default – its worst error over the chart remains several roots, because a uniform grid cannot resolve a peak that narrows without bound as a root approaches the line. All adaptive methods follow their tolerance; higher-order pairs dominate at tight tolerances, and the linearly implicit pair is dominated. The spread between the *adaptive* methods is far smaller than the gap that separates all of them from a fixed grid, which is the practical message of this section.

Appendix C.5. Narrow number formats

Since the march needs only the *phase* of $\mathcal{D}$ to a fraction of a radian, the characteristic function can even be evaluated in half precision (Table C.15). Float16 is accurate wherever no root is close to the line: after rescaling the frequency into its range it yields, on the full-HD chart of Table 9, 0.0181 % wrong counts that are *all* flagged, and 1.76 % flagged points in total; re-checking the flagged points in Float32 – in place on the device: gather, march, scatter – restores the exact chart. It does not, however, make the chart faster: at 8K the Float16 evaluation takes 1.01–1.12 times the Float32 time on all four cards (the factor of three of the T4 at full HD is an artifact of its clock control). Scalar half-precision instructions issue at the single-precision rate – a gain would require packing pairs of values into vector registers – the elementary functions are evaluated by promotion to Float32, and the conversions have a reduced throughput of their own. (An earlier version of the kernel, which passed the dual numbers through local memory, did run three times faster in Float16, `gpu/results/interactive_G4.csv`; the gain came from the smaller memory traffic and disappeared once that traffic was removed.) Float32 is therefore the production format on every card measured, and the value of the narrow formats lies in what they reveal about the method: how little

Table C.15: The discrete march with $\mathcal{D}$ evaluated in narrow number formats (400×400 chart of the fourth-order benchmark, $\omega_{\max} = 15$, best of four frequency scalings $\lambda = \Omega\mu$ per format; the march itself runs in Float32). *Flagged* points carry a sub-resolution or residual flag and are re-checked in a wider format; *failed* points did not reach $\omega_{\max}$. *Emulated: every operation is rounded to the format.

format of $\mathcal{D}$	bits (exp./mant.)	wrong Z [%]	flagged [%]	failed [%]
Float64	11/52	0.000	0.00	0.00
Float32	8/23	0.000	0.00	0.00
Float16	5/10	0.014	1.74	0.00
BFloat16*	8/7	0.171	14.05	0.00
Float8 E5M2*	5/2	8.780	98.38	3.00
Float8 E4M3*	4/3	9.086	95.89	1.59
Float4 E2M1*	2/1	100.000	100.00	10.56

precision the phase needs. BFloat16, with only seven mantissa bits, flags 14 % of the chart, and the 8-bit and 4-bit formats are useless for this purpose: with one to three mantissa bits even the *parameters* of a chart are quantized to a handful of distinct values per octave, so that 95–100 % of the points are flagged and up to one in ten counts is wrong; Float4 gets every count wrong. Load balancing, finally, does not pay: the dynamic work queue of Fig. 5, in which threads that finish early fetch new points, is 1.3–1.9 times *slower* than one thread per point on every card, because the cost per point varies only moderately and the hardware scheduler absorbs the difference at a lower overhead than the queue.

References

[1] S. A. Tobias, W. Fishwick, Theory of regenerative machine tool chatter, The Engineer 205 (1958) 199–203, 238–239, london.

[2] J. Tlusty, M. Polacek, The stability of machine tools against self-excited vibrations in machining, in: International Research in Production Engineering, ASME, 1963, pp. 465–474.

[3] Y. Altintas, E. Budak, Analytical prediction of stability lobes in milling, CIRP Annals - Manufacturing Technology 44 (1) (1995) 357–362. doi:10.1016/S0007-8506(07)62342-7.

[4] J. Munoa, X. Beudaert, Z. Dombovari, Y. Altintas, E. Budak, C. Brecher, G. Stépán, Chatter suppression techniques in metal cutting, CIRP Annals - Manufacturing Technology 65 (2) (2016) 785–808. doi:10.1016/j.cirp.2016.06.004.

[5] R. Bellman, K. L. Cooke, Differential-Difference Equations, Vol. 6 of Mathematics in Science and Engineering, Academic Press, New York, 1963.

[6] G. Stépán, Retarded Dynamical Systems: Stability and Characteristic Functions, Vol. 210 of Pitman Research Notes in Mathematics Series, Longman Scientific & Technical, Harlow, 1989, co-published with John Wiley & Sons, New York.

[7] J. K. Hale, S. M. Verduyn Lunel, Introduction to Functional Differential Equations, Vol. 99 of Applied Mathematical Sciences, Springer, New York, 1993. doi:10.1007/978-1-4612-4342-7.

[8] W. Michiels, S.-I. Niculescu, Stability, Control, and Computation for Time-Delay Systems: An Eigenvalue-Based Approach, 2nd Edition, Advances in Design and Control, SIAM, Philadelphia, PA, 2014. doi:10.1137/1.9781611973631.

[9] T. Insperger, G. Stépán, Semi-Discretization for Time-Delay Systems: Stability and Engineering Applications, Vol. 178 of Applied Mathematical Sciences, Springer, New York, 2011. doi:10.1007/978-1-4614-0335-7.

[10] R. Sipahi, S.-I. Niculescu, C. T. Abdallah, W. Michiels, K. Gu, Stability and stabilization of systems with time delay: Limitations and opportunities, IEEE Control Systems Magazine 31 (1) (2011) 38–65. doi:10.1109/MCS.2010.939135.

[11] T. Insperger, G. Stépán, Semi-discretization method for delayed systems, International Journal for Numerical Methods in Engineering 55 (5) (2002) 503–518. doi:10.1002/nme.505.

[12] T. Insperger, G. Stépán, Updated semi-discretization method for periodic delay-differential equations with discrete delay, International Journal for Numerical Methods in Engineering 61 (1) (2004) 117–141. doi:10.1002/nme.1061.

[13] T. Insperger, G. Stépán, J. Turi, On the higher-order semi-discretizations for periodic delayed systems, Journal of Sound and Vibration 313 (1–2) (2008) 334–341. doi:10.1016/j.jsv.2007.11.040.

[14] Y. Ding, L. Zhu, X. Zhang, H. Ding, A full-discretization method for prediction of milling stability, International Journal of Machine Tools and Manufacture 50 (5) (2010) 502–509. doi:10.1016/j.ijmachtools.2010.01.003.

[15] E. A. Butcher, H. Ma, E. Bueler, V. Averina, Z. Szabó, Stability of linear time-periodic delay-differential equations via Chebyshev polynomials, International Journal for Numerical Methods in Engineering 59 (7) (2004) 895–922. doi:10.1002/nme.894.

[16] D. Breda, S. Maset, R. Vermiglio, Pseudospectral differencing methods for characteristic roots of delay differential equations, SIAM Journal on Scientific Computing 27 (2) (2005) 482–495. doi:10.1137/030601600.

[17] D. Breda, S. Maset, R. Vermiglio, Stability of Linear Delay Differential Equations: A Numerical Approach with MATLAB, SpringerBriefs in Control, Automation and Robotics, Springer, New York, NY, 2015. doi:10.1007/978-1-4939-2107-2.

[18] F. A. Khasawneh, B. P. Mann, A spectral element approach for the stability of delay systems, International Journal for Numerical Methods in Engineering 87 (6) (2011) 566–592. doi:10.1002/nme.3122.

[19] Z. Wu, W. Michiels, Reliably computing all characteristic roots of delay differential equations in a given right half plane using a spectral method, Journal of Computational and Applied Mathematics 236 (9) (2012) 2499–2514. doi:10.1016/j.cam.2011.12.009.

[20] K. Engelborghs, T. Luzyanina, D. Roose, Numerical bifurcation analysis of delay differential equations using DDE-BIFTOOL, ACM Transactions on Mathematical Software 28 (1) (2002) 1–21. doi:10.1145/513001.513002.

[21] J. Sieber, K. Engelborghs, T. Luzyanina, G. Samaey, D. Roose, DDE-BIFTOOL manual — bifurcation analysis of delay differential equations, arXiv:1406.7144v4 (2016). arXiv:1406.7144.
URL https://arxiv.org/abs/1406.7144

[22] D. Breda, S. Maset, R. Vermiglio, TRACE-DDE: a tool for robust analysis and characteristic equations for delay differential equations, in: J. J. Loiseau, W. Michiels, S.-I. Niculescu, R. Sipahi (Eds.), Topics in Time Delay Systems: Analysis, Algorithms and Control, Vol. 388 of Lecture Notes in Control and Information Sciences, Springer, Berlin, Heidelberg, 2009, pp. 145–155. doi:10.1007/978-3-642-02897-7_13.

[23] R. Szalai, G. Stépán, S. J. Hogan, Continuation of bifurcations in periodic delay-differential equations using characteristic matrices, SIAM Journal on Scientific Computing 28 (4) (2006) 1301–1317. doi:10.1137/040618709.

[24] R. Szalai, Knut: a continuation and bifurcation software for delay-differential equations, source code at https://github.com/rs1909/knut, accessed 2026-07-16 (2013).
URL https://rs1909.github.io/knut/

[25] D. Bachrathy, H. T. Sykora, SemiDiscretizationMethod.jl: A Julia package to investigate the behaviour of linear delay differential equations, version 0.4.1, accessed 2026-07-16 (2026).
URL https://github.com/bachrathyd/SemiDiscretizationMethod.jl

[26] D. Bachrathy, Linear time complexity analysis of time-periodic delayed systems with multiplication free semi-discretization method, Journal of Vibration and ControlPublished online 22 May 2026 (2026). doi:10.1177/10775463261453846.

[27] J. K. Hale, S. M. Verduyn Lunel, Strong stabilization of neutral functional differential equations, IMA Journal of Mathematical Control and Information 19 (1–2) (2002) 5–23. doi:10.1093/imamci/19.1_and_2.5.

[28] W. Michiels, T. Vyhlídal, An eigenvalue based approach for the stabilization of linear time-delay systems of neutral type, Automatica 41 (6) (2005) 991–998. doi:10.1016/j.automatica.2004.11.032.

[29] S. L. Campbell, Singular Systems of Differential Equations, Vol. 40 of Research Notes in Mathematics, Pitman, San Francisco, 1980.

[30] P. Ha, V. Mehrmann, Analysis and numerical solution of linear delay differential-algebraic equations, BIT Numerical Mathematics 56 (2) (2016) 633–657. doi:10.1007/s10543-015-0577-6.

[31] J. M. Cushing, Integrodifferential Equations and Delay Models in Population Dynamics, Vol. 20 of Lecture Notes in Biomathematics, Springer, Berlin, Heidelberg, 1977. doi:10.1007/978-3-642-93073-7.

[32] I. Petráš, Fractional-Order Nonlinear Systems: Modeling, Analysis and Simulation, Nonlinear Physical Science, Springer, Berlin, Heidelberg, 2011. doi:10.1007/978-3-642-18101-6.

[33] A. R. Fioravanti, C. Bonnet, H. Özbay, S.-I. Niculescu, A numerical method for stability windows and unstable root-locus calculation for linear fractional time-delay systems, Automatica 48 (11) (2012) 2824–2830. doi:10.1016/j.automatica.2012.04.009.

[34] J. Čermák, T. Kisela, Delay-dependent stability switches in fractional differential equations, Communications in Nonlinear Science and Numerical Simulation 79 (2019) 104888. doi:10.1016/j.cnsns.2019.104888.

[35] K. Gu, V. L. Kharitonov, J. Chen, Stability of Time-Delay Systems, Birkhäuser, Boston, 2003. doi:10.1007/978-1-4612-0039-0.

[36] J. I. Neimark, D-decomposition of the space of quasi-polynomials (on the stability of linearized distributive systems), Prikladnaya Matematika i Mekhanika 13 (1949) 349–380, in Russian. English translation: American Mathematical Society Translations, Series 2, vol. 102, pp. 95–131, 1973.

[37] H. Nyquist, Regeneration theory, Bell System Technical Journal 11 (1) (1932) 126–147. doi:10.1002/j.1538-7305.1932.tb02344.x.

[38] A. V. Mikhailov, Metod garmonicheskogo analiza v teorii regulirovaniya (Method of harmonic analysis in control theory), Avtomatika i Telemekhanika (3) (1938) 27–81, in Russian.

[39] K. Walton, J. E. Marshall, Direct method for TDS stability analysis, IEE Proceedings D – Control Theory and Applications 134 (2) (1987) 101–107. doi:10.1049/ip-d.1987.0018.

[40] Z. V. Rekasius, A stability test for systems with delays, in: Proceedings of the Joint Automatic Control Conference, San Francisco, CA, 1980, paper No. TP9-A.

[41] N. Olgac, R. Sipahi, An exact method for the stability analysis of time-delayed linear time-invariant (LTI) systems, IEEE Transactions on Automatic Control 47 (5) (2002) 793–797. doi:10.1109/TAC.2002.1000275.

[42] X.-G. Li, S.-I. Niculescu, A. Çela, Analytic Curve Frequency-Sweeping Stability Tests for Systems with Commensurate Delays, Springer-Briefs in Electrical and Computer Engineering – Control, Automation and Robotics, Springer, Cham, 2015. doi:10.1007/978-3-319-15717-7.

[43] S. Yi, P. W. Nelson, A. G. Ulsoy, Time-Delay Systems: Analysis and Control Using the Lambert W Function, World Scientific, Singapore, 2010. doi:10.1142/7759.

[44] J. Louisell, A matrix method for determining the imaginary axis eigenvalues of a delay system, IEEE Transactions on Automatic Control 46 (12) (2001) 2008–2012. doi:10.1109/9.975510.

[45] T. Vyhlídal, P. Zítek, Mapping based algorithm for large-scale computation of quasi-polynomial zeros, IEEE Transactions on Automatic Control 54 (1) (2009) 171–177. doi:10.1109/TAC.2008.2008345.

[46] T. Vyhlídal, P. Zítek, QPmR – Quasi-Polynomial Root-Finder: Algorithm update and examples, in: T. Vyhlídal, J.-F. Lafay, R. Sipahi (Eds.), Delay Systems: From Theory to Numerics and Applications, Vol. 1 of Advances in Delays and Dynamics, Springer, Cham, 2014, pp. 299–312. doi:10.1007/978-3-319-01695-5_22.

[47] W.-J. Beyn, An integral method for solving nonlinear eigenvalue problems, Linear Algebra and its Applications 436 (10) (2012) 3839–3863. doi:10.1016/j.laa.2011.03.030.

[48] D. Avanessoff, A. R. Fioravanti, C. Bonnet, YALTA: a Matlab toolbox for the H_∞-stability analysis of classical and fractional systems with commensurate delays, in: 11th IFAC Workshop on Time-Delay Systems, Vol. 46 of IFAC Proceedings Volumes, 2013, pp. 839–844. doi:10.3182/20130204-3-FR-4032.00088.

[49] S.-I. Niculescu, I. Boussaada, Counting characteristic roots of linear delay differential equations. Part II: From argument principle to rightmost root assignment methods, in: D. Breda (Ed.), Controlling Delayed Dynamics: Advances in Theory, Methods and Applications, Vol. 604 of CISM International Centre for Mechanical Sciences, Springer, Cham, 2023, pp. 157–193. doi:10.1007/978-3-031-01129-0_6.

[50] D. Bachrathy, G. Stépán, Bisection method in higher dimensions and the efficiency number, Periodica Polytechnica Mechanical Engineering 56 (2) (2012) 81–86. doi:10.3311/pp.me.2012-2.01.

[51] D. Bachrathy, MDBM.jl: Multi-dimensional bisection method, version 0.1.7 (2025).
URL https://github.com/bachrathyd/MDBM.jl

[52] G. Stépán, Modelling nonlinear regenerative effects in metal cutting, Philosophical Transactions of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences 359 (1781) (2001) 739–757. doi:10.1098/rsta.2000.0753.

[53] T. Insperger, B. P. Mann, G. Stépán, P. V. Bayly, Stability of up-milling and down-milling, part 1: Alternative analytical methods, International Journal of Machine Tools and Manufacture 43 (1) (2003) 25–34. doi:10.1016/S0890-6955(02)00159-1.

[54] D. Bachrathy, G. Stépán, Improved prediction of stability lobes with extended multi frequency solution, CIRP Annals - Manufacturing Technology 62 (1) (2013) 411–414. doi:10.1016/j.cirp.2013.03.085.

[55] V. B. Kolmanovskii, A. Myshkis, Introduction to the Theory and Applications of Functional Differential Equations, Vol. 463 of Mathematics and Its Applications, Kluwer Academic Publishers, Dordrecht, 1999. doi:10.1007/978-94-017-1965-0.

[56] L. M. Delves, J. N. Lyness, A numerical method for locating the zeros of an analytic function, Mathematics of Computation 21 (100) (1967) 543–560. doi:10.1090/S0025-5718-1967-0228165-4.

[57] X. Ying, I. N. Katz, A reliable argument principle algorithm to find the number of zeros of an analytic function in a bounded domain, Numerische Mathematik 53 (1988) 143–163.

[58] P. Kravanja, M. Van Barel, Computing the Zeros of Analytic Functions, Vol. 1727 of Lecture Notes in Mathematics, Springer, Berlin, Heidelberg, 2000. doi:10.1007/BFb0103927.

[59] P. Kravanja, M. Van Barel, O. Ragos, M. N. Vrahatis, F. A. Zafiropoulos, ZEAL: A mathematical software package for computing zeros of analytic functions, Computer Physics Communications 124 (2–3) (2000) 212–232. doi:10.1016/S0010-4655(99)00429-4.

[60] T. Johnson, W. Tucker, Enclosing all zeros of an analytic function – A rigorous approach, Journal of Computational and Applied Mathematics 228 (1) (2009) 418–423.

[61] C. Hwang, Y.-C. Cheng, A numerical algorithm for stability testing of fractional delay systems, Automatica 42 (5) (2006) 825–831.

[62] B. D. Hassard, Counting roots of the characteristic equation for linear delay-differential systems, Journal of Differential Equations 136 (2) (1997) 222–235. doi:10.1006/jdeq.1996.3127.

[63] P. Zítek, T. Vyhlídal, Argument-increment based stability criterion for neutral time delay systems, in: Proceedings of the 16th Mediterranean Conference on Control and Automation, 2008, pp. 824–829. doi:10.1109/MED.2008.4602199.

[64] T. Vyhlídal, P. Zítek, Modification of Mikhaylov criterion for neutral time-delay systems, IEEE Transactions on Automatic Control 54 (10) (2009) 2430–2435. doi:10.1109/TAC.2009.2029301.

[65] Q. Xu, Z. Wang, Exact stability test of neutral delay differential equations via a rough estimation of the testing integral, International Journal of Dynamics and Control 2 (2) (2014) 154–163. doi:10.1007/s40435-013-0044-7.

[66] Q. Xu, G. Stépán, Z. Wang, Delay-dependent stability analysis by using delay-independent integral evaluation, Automatica 70 (2016) 153–157. doi:10.1016/j.automatica.2016.03.028.

[67] B. Barker, R. Nguyen, B. Sandstede, N. Ventura, C. Wahl, Computing Evans functions numerically via boundary-value problems, Physica D: Nonlinear Phenomena 367 (2018) 1–10. doi:10.1016/j.physd.2017.12.002.

[68] B. Barker, J. Humpherys, J. Lytle, K. Zumbrun, STABLAB: a MATLAB-based numerical library for Evans function computation, https://github.com/nonlinear-waves/stablab_matlab (2015).

[69] J. H. Verner, Numerically optimal Runge–Kutta pairs with interpolants, Numerical Algorithms 53 (2–3) (2010) 383–396. doi:10.1007/s11075-009-9290-3.

[70] C. Tsitouras, Runge–Kutta pairs of order 5(4) satisfying only the first column simplifying assumption, Computers & Mathematics with Applications 62 (2) (2011) 770–775. doi:10.1016/j.camwa.2011.06.002.

[71] C. Rackauckas, Q. Nie, DifferentialEquations.jl – a performant and feature-rich ecosystem for solving differential equations in Julia, Journal of Open Research Software 5 (1) (2017) 15. doi:10.5334/jors.151.

[72] J. Revels, M. Lubin, T. Papamarkou, Forward-mode automatic differentiation in Julia (2016). arXiv:1607.07892.
URL https://arxiv.org/abs/1607.07892

[73] J. Bezanson, A. Edelman, S. Karpinski, V. B. Shah, Julia: A fresh approach to numerical computing, SIAM Review 59 (1) (2017) 65–98. doi:10.1137/141000671.

[74] J. M. Tribolet, A new phase unwrapping algorithm, IEEE Transactions on Acoustics, Speech, and Signal Processing 25 (2) (1977) 170–177.

[75] L. Zhang, G. Stépán, Exact stability chart of an elastic beam subjected to delayed feedback, Journal of Sound and Vibration 367 (2016) 219–232. doi:10.1016/j.jsv.2016.01.002.

[76] W. Michiels, Spectrum-based stability analysis and stabilisation of systems described by delay differential algebraic equations, IET Control Theory & Applications 5 (16) (2011) 1829–1842. doi:10.1049/iet-cta.2010.0752.

[77] S. G. Johnson, QuadGK.jl: Gauss–Kronrod integration in Julia, version 2.11.3 (2013).
URL https://github.com/JuliaMath/QuadGK.jl

[78] D. J. VandenHeuvel, DelaunayTriangulation.jl: A Julia package for Delaunay triangulations and Voronoi tessellations in the plane, Journal of Open Source Software 9 (101) (2024) 7174. doi:10.21105/joss.07174.

[79] M. Kidd, G. Stépán, Delayed control of an elastic beam, International Journal of Dynamics and Control 2 (1) (2014) 68–76. doi:10.1007/s40435-014-0079-4.

[80] Z. Gao, L.-R. Zhai, Y.-D. Liu, Robust stabilizing regions of fractional-order PI$^{\lambda}$ controllers for fractional-order systems with time-delays, International Journal of Automation and Computing 14 (3) (2017) 340–349. doi:10.1007/s11633-015-0941-7.

[81] S. E. Hamamci, An algorithm for stabilization of fractional-order time delay systems using fractional-order PID controllers, IEEE Transactions on Automatic Control 52 (10) (2007) 1964–1969. doi:10.1109/TAC.2007.906243.

[82] N. Hohenbichler, Comments on "An algorithm for stabilization of fractional-order time delay systems using fractional-order PID controllers", IEEE Transactions on Automatic Control 53 (11) (2008) 2712. doi:10.1109/TAC.2008.2007535.

[83] A. Ruszewski, Stability regions of closed loop system with time delay inertial plant of fractional order and fractional order PI controller, Bulletin of the Polish Academy of Sciences: Technical Sciences 56 (4) (2008) 329–332.

[84] Y.-C. Cheng, C. Hwang, Stabilization of unstable first-order time-delay systems using fractional-order PD controllers, Journal of the Chinese Institute of Engineers 29 (2) (2006) 241–249. doi:10.1080/02533839.2006.9671121.

[85] H. H. Rosenbrock, Some general implicit processes for the numerical solution of differential equations, The Computer Journal 5 (4) (1963) 329–330. doi:10.1093/comjnl/5.4.329.

[86] L. F. Shampine, M. W. Reichelt, The MATLAB ODE suite, SIAM Journal on Scientific Computing 18 (1) (1997) 1–22. doi:10.1137/S1064827594276424.

[87] E. Hairer, G. Wanner, Solving Ordinary Differential Equations II: Stiff and Differential-Algebraic Problems, 2nd Edition, Vol. 14 of Springer Series in Computational Mathematics, Springer, Berlin, Heidelberg, 1996. doi:10.1007/978-3-642-05221-7.